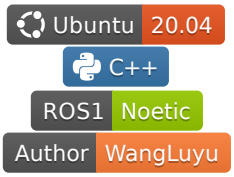
导航考核

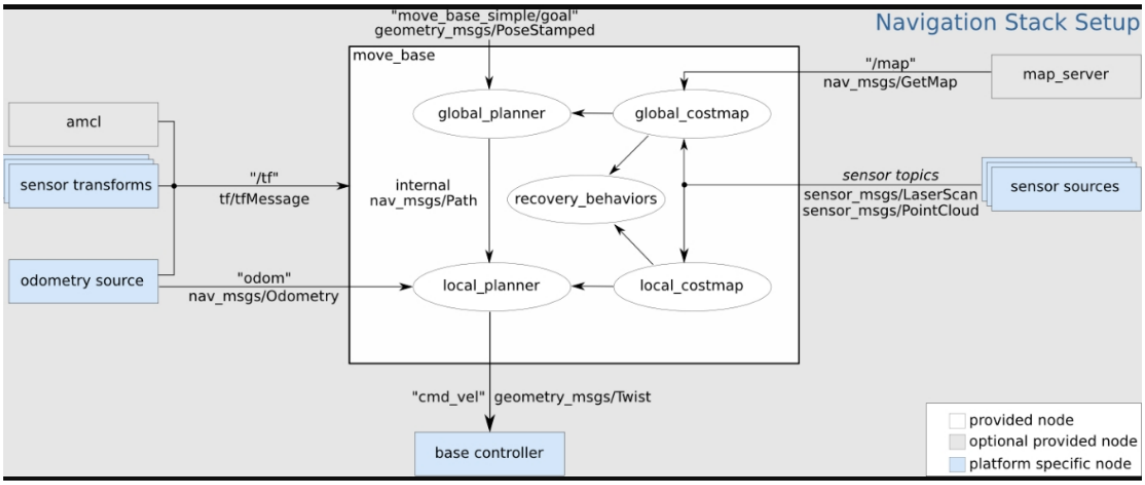


考核一
ROS Navigation Stack
问题：
考核二 实现二维建图
1.URDF (Universal Robot Description Format)
2.xacro
🤖**实现一个简单的带激光雷达的机器人：**
3.gmapping+二维建图方法汇总
4.Gazebo仿真测试
利用官方world地图+turtlebot3 运行：
自己建图+ turtlebot3：🗺️
自己建机器人+自己的图：
问题：
考核3 实现机器人自主定位和导航
1.航点导航
基于turtlebot3 的航点导航
自定义机器人+自定义地图航点导航
move_base原理
2.自主探索算法
Frontier-based Exploration
三种软件包实现
基于turtlebot3实现explore_lite自主建图
自定义机器人实现explore_lite自主建图
考核四：实现三维建图
1.FAST-LIO
原理
2.复现
3.更加适配动态的建图算法
RF-LIO
Fast_LIO+YOLO
4.利用高维地图实现导航
总结
学到了什么🐾
遇到的问题以及解决方法❔
AI使用情况
附件：

> 文档内含四个任务的学习笔记，字数较多，但它也见证了我这一路走来的成长。请各位学长移步正文进行验收，劳烦各位学长指正！🎉😸

考核一

ROS Navigation Stack



1. **全局地图map_server**：提供静态地图
    - 地图来源：1.通过SLAM提供实时地图 2.提供SLAM提前建立并保存或者通过其他方式制作的地图
    - `/static_map` 服务：响应 `nav_msgs/GetMap` 请求，返回地图的相关信息
    - `/map` 话题： **循环发布**静态地图，其他模块（如 `amcl` 估算机器人的位置、`move_base` 构建全局代价地图）订阅此话题以使用地图；**消息类型**：`nav_msgs/OccupancyGrid`
2. **自身定位**
    - **AMCL**：**提供准确机器人的定位**；
        - 通过 `map -> odom` 坐标变换来修正里程计的漂移误差（odom节点发布 `odom -> base_link` 的坐标变换,AMCL节点发布`map -> odom` 的坐标变换，这两者叠加)；
        - 原理：自适应蒙特卡罗定位，粒子滤波算法；
        - 应用：`move_base` 使用 AMCL 发布的位姿信息来进行路径规划
    - **sensor transforms**:**传感器坐标变化**;
        - 利用 TF 将传感器数据从传感器原始坐标系转换到转换到 `base_link` 或 `map` 坐标系，如从`laser_frame`坐标系转到`base_link`；
        - 应用：costmap构建时需要传感器数据；amcl匹配位置时需要转换；planner计算路径
        - map → odom → base_link → sensor_frame
    - **odometry source：提供位移向量和原点位置信息，估计机器人相对起始位置的运动**
        - 使用tf和`nav_msgs/Odometry`消息发布的里程信息，发布`odom → base_link`坐标变换，表示机器人相对运动
        - 存在问题：里程计通常通过 **轮速计** 或 **IMU** 数据估算机器人位移，可能出现误差累积（漂移），因此需要使用AMCL修正；
        - 应用：`local_planner` 使用 `odom`提供的 `odom -> base_link`变换来确定机器人的当前位置，然后基于此信息计算局部路径和控制命令；
3. **路径规划**

move_base： 导航栈的核心；负责接受目标点+调用global_planner和local_planner+输出速度指令`/cmd_vel`

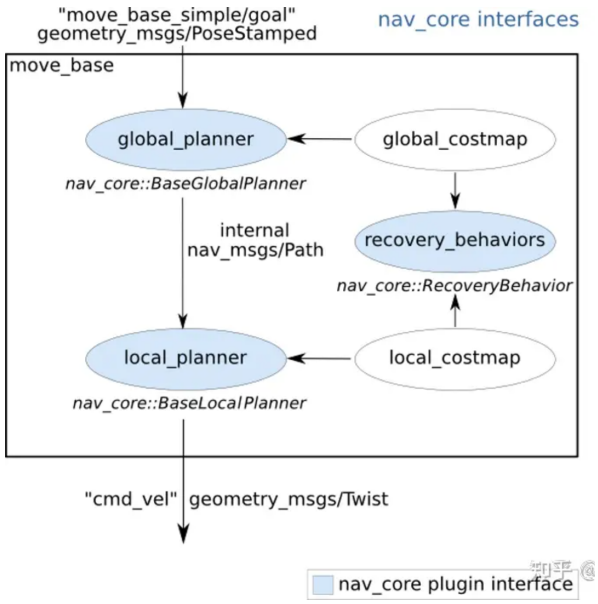
- 工作流程：

  用户通过 `move_base_simple/goal` 话题或 `move_base` 的 `MoveBaseAction` 接口发送目标位置（`Pose`）；目标位置由 `move_base` 接收，作为一个 **2D 目标位姿**（`geometry_msgs/PoseStamped`）；

  `global_planner` 根据 `global costmap` 计算当前位姿到目标的全局路径（避开了costmap障碍物但是不考虑动态障碍物）；

  `local_planner` 使用 `local costmap` 计算一个动态实时的路径,同时根据机器人的速度和动态约束生成控制指令驱动机器人；

  最后，`move_base` 将 `local_planner` 生成的速度指令通过 `/cmd_vel` 话题发送给机器人控制系统，驱动机器人移动



- 两个插件：
  - **global_planner 全局路径规划**：输入起点+终点+全局costmap，输出一条全局路径；常用实现方法：**Dijkstra/A***
  - **local_planner 局部路径规划**: 输入局部costmap+起点+终点+机器人状态位姿；常用算法：**DWA（动态窗口）/TEB**；输出：速度指令（geometry_msgs/Twist）
- 两个对象：

> **costmap_2d(代价地图)：**
>
> 表示环境中的障碍物和可通行区域，并赋予不同区域不同的代价值（例如，障碍物周围的区域代价值较高）；
>
> 代价地图通常由多个层组成： **Obstacle Map Layer（障碍物层）**：根据传感器数据设置代价高的区域为障碍物； **Inflation Layer（膨胀层）**：扩展障碍物的影响范围，把障碍物区域扩展成较大的缓冲区；

-- **global costmap:**主要表示静态环境，如障碍物;长期路径

-- **local costmap:**为 **Local Planner** 提供参考，发现并绕过地图上没有的**动态障碍物**（如行人、移动的小车）

- **recovery_behaviors 恢复行为**

  机器人在遇到障碍、困境或路径阻塞时，通过特定的策略恢复其导航能力，避免机器人无限期地停滞不前；在局部和全局planner失败下激活

MoveBase类

```
class MoveBase {
  public:
    MoveBase(tf2_ros::Buffer& tf);
    virtual ~MoveBase();
    // Actionlib 的回调函数，当收到新目标时触发
    void executeCb(const move_base_msgs::MoveBaseGoalConstPtr& move_base_goal);
  private:
    // 1. 核心插件指针
    boost::shared_ptr<nav_core::BaseGlobalPlanner> planner_;
    boost::shared_ptr<nav_core::BaseLocalPlanner> tc_;
    // 2. 插件加载器
    pluginlib::ClassLoader<nav_core::BaseGlobalPlanner> bgp_loader_;
    pluginlib::ClassLoader<nav_core::BaseLocalPlanner> blp_loader_;
    // 3. 代价地图对象
    costmap_2d::Costmap2DROS* planner_costmap_ros_;
    costmap_2d::Costmap2DROS* controller_costmap_ros_;
    // 4. 核心控制循环函数
    bool executeCycle(geometry_msgs::PoseStamped& goal);
    void planThread(); // 独立线程负责全局规划
};
// 控制循环
bool MoveBase::executeCycle(geometry_msgs::PoseStamped& goal)
{
    // 执行全局规划
    planner_->makePlan(current_pose, goal, global_plan_);
    // 执行局部规划
    tc_->computeVelocityCommands(cmd_vel_);
    return true;
}
```

nav_core类:纯虚函数，定义接口

BaseGlobalPlanner 和 BaseLocalPlanner

```
//global_planner接口
namespace nav_core {
  class BaseGlobalPlanner {
    public:
      //根据起点，目标规划路径
      virtual bool makePlan(const geometry_msgs::PoseStamped& start,const
geometry_msgs::PoseStamped& goal,std::vector<geometry_msgs::PoseStamped>& plan) =
0;
      //初始化代价地图指针等插件
      virtual void initialize(std::string name,costmap_2d::Costmap2DROS*
costmap_ros) = 0;
      virtual ~BaseGlobalPlanner(){}
    protected:
      BaseGlobalPlanner(){}
  };
}

//局部规划接口
```

```
namespace nav_core {
  class BaseLocalPlanner {
    public:
      //计算当前时刻的速度命令
      virtual bool computeVelocityCommands(geometry_msgs::Twist& cmd_vel) = 0;
      // 检查路径是否有效
      virtual bool isGoalReached() = 0;
      // 接收来自全局规划器的路径
      virtual bool setPlan(const std::vector<geometry_msgs::PoseStamped>& plan) =
0;
      //初始化
      virtual void initialize(std::string name, tf2_ros::Buffer* tf,
costmap_2d::Costmap2DROS* costmap_ros) = 0;
      virtual ~BaseLocalPlanner(){}
    protected:
      BaseLocalPlanner(){}
  };
}
class RecoveryBehavior
{
public:
  virtual void runBehavior() = 0;
};
```

4. **运动控制 base controller**： 接受速度指令

   接受来自`move_base`的速度指令`/cmd_vel（geometry_msgs/Twist）`

   订阅`cmd_vel`话题， 将该话题上的速度命令`(vx, vy, vtheta`转换为电机命令`(cmd_vel.linear.x, cmd_vel.linear.y, cmd_vel.angular.z)`发送给移动基座

5. **环境感知 sensor sources**：提供环境数据

   常见输入：来自传感器的消息：激光雷达，点云，超声`sensor_msgs/LaserScan`消息或者`sensor_msgs/PointCloud`消息

- odom ：

  话题： /odom (nav_msgs/Odometry)

  来源：base controller

6. **建图算法：**
   - **DWA**（Dynamic Window Approaches）：**局部路径规划算法**
     - 原理：速度采样-->轨迹预测（推算）-->轨迹评价

       确定动态窗口，窗口内**采集机器人速度样本（v,w）**，假设未来短时间内保持这个速度运行，**预测（动力学模型）**模拟出运动轨迹， 并对他们打分（有一个评价函数）， 选出得分最高的，机器人按照最优轨迹运动，不断寻哈un；
     - 优点：实时避障，实时性号好，适合动态环境下的实时避障
     - 缺点：陷入局部最优解，对障碍物密集环境响应慢（环境拥挤会反复计算速度并且每次计算后找最优解，导致响应慢），对传感器噪声敏感，过于保守（出现过度回避绕远）
   - **Dijkstra**：计算单源节点到其他所有节点的最短路径，采用贪心算法

- **D***：**全局路径规划算法**+环境变化时对路径更新
    - 原理：初始化-->增量更新

      对A*算法的改进：**A *算法：**使用优先队列扩展，每个节点有两个主要代价函数（起点到当前的实际，当前到目标的估计），搜索代价最小的节点；而D * 算法增加了实时更新节点的代价

      从目标点开始，目标的代价是0，反推计算每个节点的代价值（即到目标点的最短路径成本），然后反向传播到起点；当 **障碍物** 出现或 **目标点** 改变时，D* 会从发生变化的节点开始，更新路径代价，并重新计算路径
    - 优点：动态更新路径，适应性强（实时作出规划）
    - 缺点：对地图精度要求高，复杂难算

问题：

假设某机器人分别在**狭窄通道**和**动态环境**中运动，请你分析DWA算法和A*算法分别适用于哪种场景，并说明原因。如果要设计一种混合导航框架，请简述设计思路。

解答：

1. **狭窄通道：DWA；**DWA实时性强，能快速计算运动指令避开障碍物，短时间作出反应； D*主要计算的是全局最优路径，核心基于全局优化，在狭窄通道中障碍物多，他会从头开始重新评估整个路径，导致响应速度慢，不擅长在短时间内作出快速的决策，并且计算开销大
2. **动态环境：D*；**D * 适合于全局路径规划，且能够根据实时环境变化动态更新，对路径迅速调整；DWA主要依赖局部障碍物信息，通过局部的窗口评估接下来选择的路径，对大规模的动态环境变化不敏感，没有全局的考虑，可能出现局部最优解而非全局最优解
3. **混合导航框架**：**D*算法用于全局路径规划，同时DWA算法处理局部避障；**

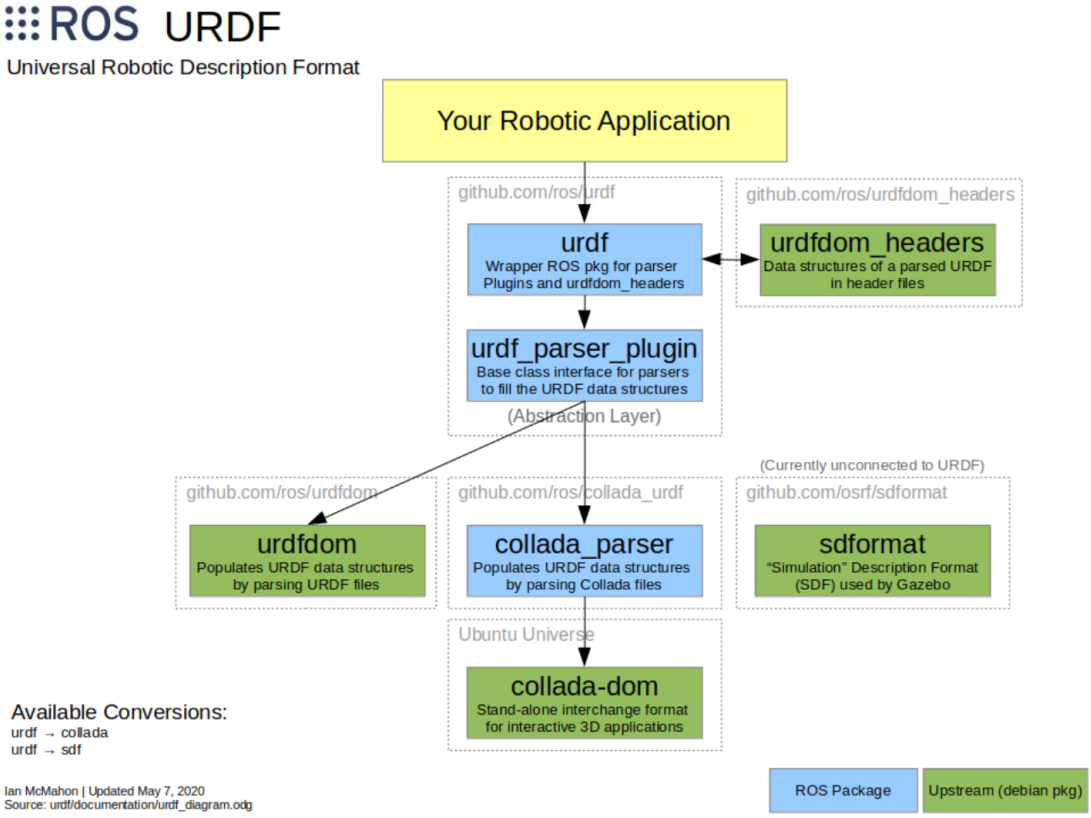
D*负责根据环境和目标计算出一个全局最优路径，当机器人接近障碍物或遇到动态变化的障碍时候，使用DWA计算出最优控制指令避障；如果机器人在全局路径没有遇到障碍物，那么继续走，遇到障碍物触发DWA，如果障碍物在全局路径的关键节点（比如刚好在出口），触发D *重新规划；

考核二 实现二维建图

1.URDF (Universal Robot Description Format)

> 通用机器人描述格式，包含的内容有：连杆、关节，运动学和动力学参数、可视化模型、碰撞检测模型等

框架如下：



urdf文件格式

完整的机器人示例:

```
    <?xml version="1.0"?>
<robot name="simple_robot">   //1.

    <link name="base_link">     //2.
        <visual>
            <geometry>
                <cylinder length="0.6" radius="0.2"/>
            </geometry>
            <material name="blue"/>
        </visual>
        <collision>
            <geometry>
                <cylinder length="0.6" radius="0.2"/>
            </geometry>
        </collision>
        <inertial>
            <mass value="1.0"/>
            <origin xyz="0 0 0"/>
            <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.1"/>
        </inertial>
    </link>

    <joint name="joint1" type="revolute">       //3.
        <parent link="base_link"/>
        <child link="link1"/>
        <origin xyz="0 0 0.6" rpy="0 0 0"/>
        <axis xyz="0 0 1"/>
        <limit lower="-1.57" upper="1.57" effort="1" velocity="1"/>
    </joint>
```

```
    <link name="link1">
        <visual>
            <geometry>
                <box size="0.5 0.1 0.1"/>
            </geometry>
            <material name="green"/>
        </visual>
        <collision>
            <geometry>
                <box size="0.5 0.1 0.1"/>
            </geometry>
        </collision>
    </link>
</robot>

1.link :描述机器人各单部分，如轮子
2.visual  :描述可视化外观：盒子box 球sphere
3.collision  :描述碰撞模型
4.inerial  :描述惯性属性和质量
    -包含以下属性：
    <mass value="1.0"/>
    <origin xyz="0 0 0"/>
    <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.1"/>
    mass 定义质量，inertia 定义惯性矩阵，origin 定义惯性计算的位置
5.material:描述颜色和材质
    <material name="blue">
    <color rgba="0 0 1 1"/>
6.joint:机器人链接之间的连接关系，每个joint连接2个link
    基本结构：
<joint name="joint_name" type="revolute">
    <parent link="parent_link"/>
    <child link="child_link"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>     关节相对于父link的位置朝向
    <axis xyz="0 0 1"/>      指定关节的旋转轴或平移方向
    <limit lower="-1.57" upper="1.57" effort="1" velocity="1"/>
    关节的运动限制，包括 lower 和 upper 角度范围、effort（最大力矩）和 velocity（最大速度）
常见type:revolute旋转关节;prismatic平移关节;fixed固定关节
```

2.xacro

> 简化和优化URDF，URDF宏语言

```
1.property以及property块

可以插入xml任何地方，定义变量，支持在其他地方引用。
    引用：通过 ${} 的方式获取其值
    <xacro:property name="link_length" value="0.5"/>
    <xacro:property name="front_left_origin">
        <origin xyz="0.3 0 0" rpy="0 0 0" />
    </xacro:property>
引用：<circle diameter="${2 * radius}" />

2.条件判断if/unless(条件为假时执行)
    <xacro:if value="${use_wheel}">
```

```
        <xacro:wheel radius="0.5" width="0.2"/>
    </xacro:if>

3.定义宏<xacro:macro> 定义可复用的模板

  <xacro:macro name="pr2_caster" params="suffix *origin **content
**anothercontent">
  <joint name="caster_${suffix}_joint">
        <axis xyz="0 0 1" />
  </joint>
  <link name="caster_${suffix}">
        <xacro:insert_block name="origin" />
        <xacro:insert_block name="content" />
        <xacro:insert_block name="anothercontent" />
  </link>
</xacro:macro>

   # 参数: suffix *origin（*表示为块参数）**content **anothercontent（**表示允许插入任意
数量的元素）
   # 使用: 例: <xacro:wheel radius="0.5" width="0.2"/>调用会生成一个名为 wheel_link 的
link
   # 宏可以包含其他宏:
        <xacro:macro name="bar" params="y">
            <in_bar>
                <xacro:foo x="${y}" />
            </in_bar>
      </xacro:macro>

   # 可以有默认参数
         <xacro:macro name="foo" params="x:=${x} y:=${2*y} z:=0"/>
     上面的写法如x，还可以写成如  x:=^ y:=^|expr_b
         + ^表示使用同名的外部范围属性
         + 管道 |表示如果该属性在外部范围中未定义，则使用给定的备选
4.include:引入其他xacro文件
   <xacro:include filename="$(myrobot)/urdf/wheel.xacro"/>路径
5.arg:获取命令行传入的参数
    <xacro:arg name="wheel_radius" default="0.3"/>
    命令行: roslaunch myrobot display.launch wheel_radius:=0.4
6.支持大多数 rospack 命令
    如:
        <foo value="$(find xacro)" />
7.带有动态定义名称的元素或属性:
        <xacro:element xacro:name="${element_name}" [other attributes]>
            [content]
        </xacro:element>
        <xacro:attribute name="${attribute_name}" value="${attribute_value}"/>
8.处理顺序: xacro 首先加载所有 include，然后处理所有属性和宏定义，最后实例化宏并评估表达式。因
此，后续的属性或宏定义会覆盖之前的定义。
```

工具

check_urdf：命令行工具check_urdf尝试解析文件为URDF描述，并打印生成的运动链描述或错误信息

使用：进入目录，check_urdf myfirst.urdf

urdf_to_graphiz：生成可视化图形，例：urdf_to_graphiz myfirst.urdf，生成multipleshapes.gv和multipleshapes.pd

🤖实现一个简单的带激光雷达的机器人：

参考网络上的教程效仿，如下：

名字叫marsbot，一共有三个轮子，前一后二；身体是红色，轮子是黑色，雷达是蓝色；惯性参数和摩擦力设置在实际仿真建图过程中询问AI不断调整；添加差速驱动插件和激光雷达插件；后续使用它分别完成建图和航点导航任务；

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="marsbot">

    <material name="blue"><color rgba="0 0 0.8 1"/></material>
    <material name="black"><color rgba="0 0 0 1"/></material>
    <material name="red"><color rgba="1 0 0 1"/></material>

    <link name="base_link">
        <visual>
            <geometry><box size="0.3 0.2 0.1"/></geometry>
            <material name="red"/>
        </visual>
        <collision>
            <geometry><box size="0.3 0.2 0.1"/></geometry>
        </collision>
        <inertial>
            <mass value="5.0"/>
            <origin xyz="0 0 0" rpy="0 0 0"/>
            <inertia ixx="0.05" ixy="0" ixz="0" iyy="0.05" iyz="0" izz="0.05"/>
        </inertial>
    </link>

    <link name="left_wheel">
        <visual>
            <geometry><cylinder length="0.04" radius="0.05"/></geometry>
            <material name="black"/>
        </visual>
        <collision>
            <geometry><cylinder length="0.04" radius="0.05"/></geometry>
        </collision>
        <inertial>
            <mass value="0.5"/>
            <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0"
izz="0.001"/>
        </inertial>
    </link>

    <gazebo reference="left_wheel">
        <mu1>200.0</mu1>
        <mu2>200.0</mu2>
        <kp>1000000.0</kp>
        <kd>10.0</kd>
        <material>Gazebo/Black</material>
    </gazebo>
```

```
    <joint name="left_wheel_joint" type="continuous">
        <parent link="base_link"/>
        <child link="left_wheel"/>
        <origin xyz="-0.1 0.12 -0.05" rpy="1.5707 0 0"/>
        <axis xyz="0 0 -1"/>
    </joint>

    <link name="right_wheel">
        <visual>
            <geometry><cylinder length="0.04" radius="0.05"/></geometry>
            <material name="black"/>
        </visual>
        <collision>
            <geometry><cylinder length="0.04" radius="0.05"/></geometry>
        </collision>
        <inertial>
            <mass value="0.5"/>
            <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0"
izz="0.001"/>
        </inertial>
    </link>

    <gazebo reference="right_wheel">
        <mu1>200.0</mu1>
        <mu2>200.0</mu2>
        <kp>1000000.0</kp>
        <kd>10.0</kd>
        <material>Gazebo/Black</material>
    </gazebo>

    <joint name="right_wheel_joint" type="continuous">
        <parent link="base_link"/>
        <child link="right_wheel"/>
        <origin xyz="-0.1 -0.12 -0.05" rpy="1.5707 0 0"/>
        <axis xyz="0 0 -1"/>
    </joint>

    <link name="caster_wheel">
        <visual>
            <geometry><sphere radius="0.05"/></geometry>
            <material name="black"/>
        </visual>
        <collision>
            <geometry><sphere radius="0.05"/></geometry>
        </collision>
        <inertial>
            <mass value="0.1"/>
            <inertia ixx="0.0001" ixy="0" ixz="0" iyy="0.0001" iyz="0"
izz="0.0001"/>
        </inertial>
    </link>

    <joint name="caster_joint" type="fixed">
        <parent link="base_link"/>
        <child link="caster_wheel"/> <origin xyz="0.12 0 -0.05" rpy="0 0 0"/>
    </joint>

    <gazebo reference="caster_wheel">
```

```
        <mu1>0.001</mu1>
        <mu2>0.001</mu2>
        <material>Gazebo/Black</material>
    </gazebo>

    <link name="lidar_link">
        <visual>
            <geometry><cylinder length="0.05" radius="0.05"/></geometry>
            <material name="blue"/>
        </visual>
        <collision>
            <geometry><cylinder length="0.05" radius="0.05"/></geometry>
        </collision>
        <inertial>
            <mass value="0.1"/>
            <inertia ixx="0.0001" ixy="0" ixz="0" iyy="0.0001" iyz="0"
izz="0.0001"/>
        </inertial>
    </link>

    <joint name="lidar_joint" type="fixed">
        <parent link="base_link"/>
        <child link="lidar_link"/>
        <origin xyz="0.1 0 0.075" rpy="0 0 3.14159"/>
    </joint>

    <gazebo>
        <plugin name="differential_drive_controller"
filename="libgazebo_ros_diff_drive.so">
            <updateRate>50.0</updateRate>
            <leftJoint>left_wheel_joint</leftJoint>
            <rightJoint>right_wheel_joint</rightJoint>
            <wheelSeparation>0.24</wheelSeparation>
            <wheelDiameter>0.1</wheelDiameter>
            <wheelTorque>500</wheelTorque>
            <wheelAcceleration>2.0</wheelAcceleration>
            <commandTopic>cmd_vel</commandTopic>
            <odometryTopic>odom</odometryTopic>
            <robotBaseFrame>base_link</robotBaseFrame>
            <publishWheelTF>false</publishWheelTF>
            <publishWheelJointState>true</publishWheelJointState>
            <publishTf>true</publishTf>
            <odometrySource>encoder</odometrySource>
        </plugin>
    </gazebo>

    <gazebo reference="lidar_link">
        <sensor type="ray" name="laser">
            <pose>0 0 0 0 0 0</pose>
            <visualize>true</visualize>
            <update_rate>40</update_rate>
            <ray>
                <scan>
                    <horizontal>
                        <samples>720</samples>
                        <resolution>1</resolution>
                        <min_angle>-3.14159</min_angle>
                        <max_angle>3.14159</max_angle>
```

```
                </horizontal>
            </scan>
            <range>
                <min>0.15</min>
                <max>30.0</max>
                <resolution>0.01</resolution>
            </range>
        </ray>
        <plugin name="gazebo_ros_head_hokuyo_controller"
filename="libgazebo_ros_laser.so">
            <topicName>/scan</topicName>
            <frameName>lidar_link</frameName>
        </plugin>
    </sensor>
</gazebo>

</robot>
```

3.gmapping+二维建图方法汇总

> Gmapping是一个基于**2D激光雷达**使用RBPF（Rao-Blackwellized Particle Filters）算法完成**二维栅格地图**构建的SLAM算法

- 原理：
    - 采用了**RB粒子滤波器（RBPF）**，将复杂的SLAM问题（同时定位与建图）分解为定位和建图两个相对独立的过程，先进行定位（估计机器人轨迹），在已知轨迹的基础上再进行建图；每个粒子独立维护一张地图（路径的历史假设+基于路径假设获得的地图+权重），最终全局地图由权重最高的粒子产生；
    - 算法流程：

      采样（基于odom信息和上一时刻的位姿预测当前的位姿，并且加上一定随机噪声以得到不同的预测结果）-->更新（在该粒子预测的位姿的小领域内寻找一个与激光数据匹配度最高的位姿）--->计算权重（实际的激光数据与假设预测得到的地图匹配度越高，权重越大）--->选择性重采样（有效粒子数小于阈值时才进行[避免粒子退化]；淘汰权重低的粒子，复制权重高的粒子）

      以上步骤对每一个粒子循环，最终更新栅格地图
- 激光雷达：包括固定底盘和可旋转的头部结构，头部有红外激光发射器和接受器，通过计时器记录时长就能算出到障碍物的距离，然后不停旋转，刷新障碍物的分布
- **2D激光SLAM算法汇总**：

  Gmapping：RBPF
    - 小场景下非常稳定，能够实时构建栅格地图；环境增大，所需的粒子数量呈指数级增加，内存不足

  CoreSLAM：简单的距离加权和随机扰动来模拟机器人的移动
    - 它首先对当前机器人的位姿估计施加一系列**随机微小扰动**（模拟粒子滤波的简化版），产生多个候选位姿；接着，针对每个候选位姿计算激光扫描点与当前栅格地图的**匹配得分**，得分基于点到已知障碍物的距离加权和；
    - 算法直接选取得分最高的候选位姿作为当前时刻的估计值，并立即更新地图
    - **极低**的计算开销，代码极其精简；精度低，没有回环检测，在大场景下漂移严重

  Hector slam：高斯-牛顿迭代法

通过将当前激光扫描点云与已有地图进行直接配准来求解机器人的最优位姿

- 不需要里程计，更新频率高，适合无人机；要求激光雷达频率极高

KartoSLAM：

- 将机器人的位姿看作图的节点，将位姿间的约束看作边，利用稀疏矩阵进行全局一致性优化
- **全局回环检测**能力稳定，适合中大型室内环境、需要高精度长距离建图的场

LagoSLAM

- 将机器人的位姿视为图的节点，里程计、雷达等视为边，忽略坐标，只提取边里的旋转角度信息，通过线性最小二乘法，在全局范围内计算出每个节点的最优方向；确定了所有节点的角度后，再运行一次线性最小二乘，求出每个节点在世界坐标系下的精确坐标；
- 适合中大型静态环境，不需要初值来预测；但是无法处理传感器本身带来噪声

Cartographer：

- 传感器数据进入后，利用 IMU 和里程计预测，之后用雷达数据和子图匹配；当子图累积了足够多的扫描数据后，它不再更改，继续下一个空子图建图，同时用当前的扫描数据在所有旧子图中搜索，看机器人是否回到以前位置
- 回环检测强

对比：（AI生成）

算法名称	核心原理	适用场景	优点	缺点
Gmapping	**RBPF (改进粒子滤波)**	**小型室内、平坦地面,** 计算资源充足的上位机。	极度成熟，对激光雷达频率要求低，开箱即用。	内存随地图增大指数级增长；无回环检测，**大场景易偏**。
Cartographer	**局部子图 + 全局分枝定界优化**	工业级大型场景、多层建筑、需要极高可靠性的项目。	**回环检测最强**；支持多传感器融合（IMU+Lidar）；支持地图扩展。	参数极其复杂（上百个），对 CPU 要求高，调试难度大。
LagoSLAM	**线性图优化 (Linear Approximation)**	研究图优化初始化、**中大型静态环境**。	**无需初始位姿估计**；计算速度比非线性优化（如 GTSAM）快，不易陷入局部最优。	属于“批处理”式优化，实时性不如前端匹配；对非线性误差处理有限。
Hector SLAM	**高斯-牛顿扫描匹配 (Scan-to-Map)**	**无人机、无里程计的移动平台、高频率雷达设备**。	**无需里程计**；更新频率极高，适合姿态剧烈变化的仿真。	极易漂移；对雷达性能要求高（频率需 > 20Hz）；无回环。
Karto SLAM	**基于稀疏矩阵的图优化 (SPA)**	中型办公环境、对路径平滑度有要求的移动机器人。	具备基础回环检测，比 Gmapping 节省内存，精度较高。	后端优化压力随地图增大线性增加；插件依赖较多。
CoreSLAM	**随机扰动采样 (极简版 PF)**	极低成本嵌入式芯片（如 ESP32/单片机）。	代码不到 200 行，运行速度极快。	精度差，无回环，仅适合演示教学。

- **ros包的结构和功能**

	名称	类型	描述
Topic 订阅	tf	tf/tfMessage	用于激光雷达坐标系，基坐标系，里程计坐标系之间的变换
	scan	sensor_msgs/LaserScan	激光雷达扫描数据
Topic 发布	map_metadata	nav_msgs/MapMetaData	发布地图Meta数据
	map	nav_msgs/OccupancyGrid	发布地图栅格数据
	~entropy	std_msgs/Float64	发布机器人姿态分布熵的估计
Service	dynamic_map	nav_msgs/GetMap	获取地图数据

知乎 @小白学移动机器人

	TF变换	描述
必需的TF变换	<scan frame> → base_link	激光雷达坐标系与基坐标系之间的变换，一般由robot_state_publisher或static_transform_publisher发布
	base_link → odom	基坐标系与里程计坐标系之间的变换，一般由里程计节点发布
发布的TF变换	map → odom	地图坐标系与机器人里程计坐标系之间的变换，描述机器人在地图中的位姿

`map_metadata(nav_msgs/mapMetaData)` 地图信息:加载时间，分辨率，宽高，原点坐标

`map(nav_msgs/OccupancyGrid)` 栅格地图数据

`~entropy(std_msgs/Float64)` 机器人定位的置信度：该值越大，出错可能性越大

4.Gazebo仿真测试

利用官方world地图+turtlebot3 运行：

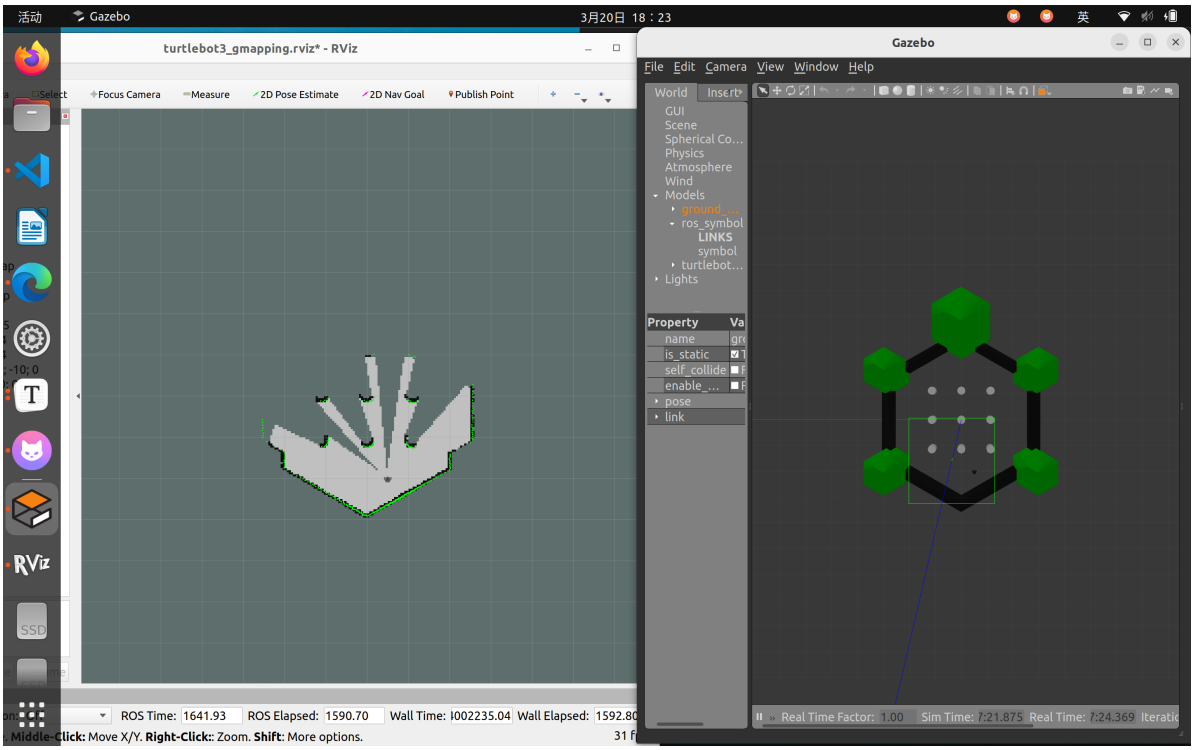
启动Gazebo：`roslaunch turtlebot3_gazebo turtlebot3_world.launch`

gmapping 建图：`roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping`

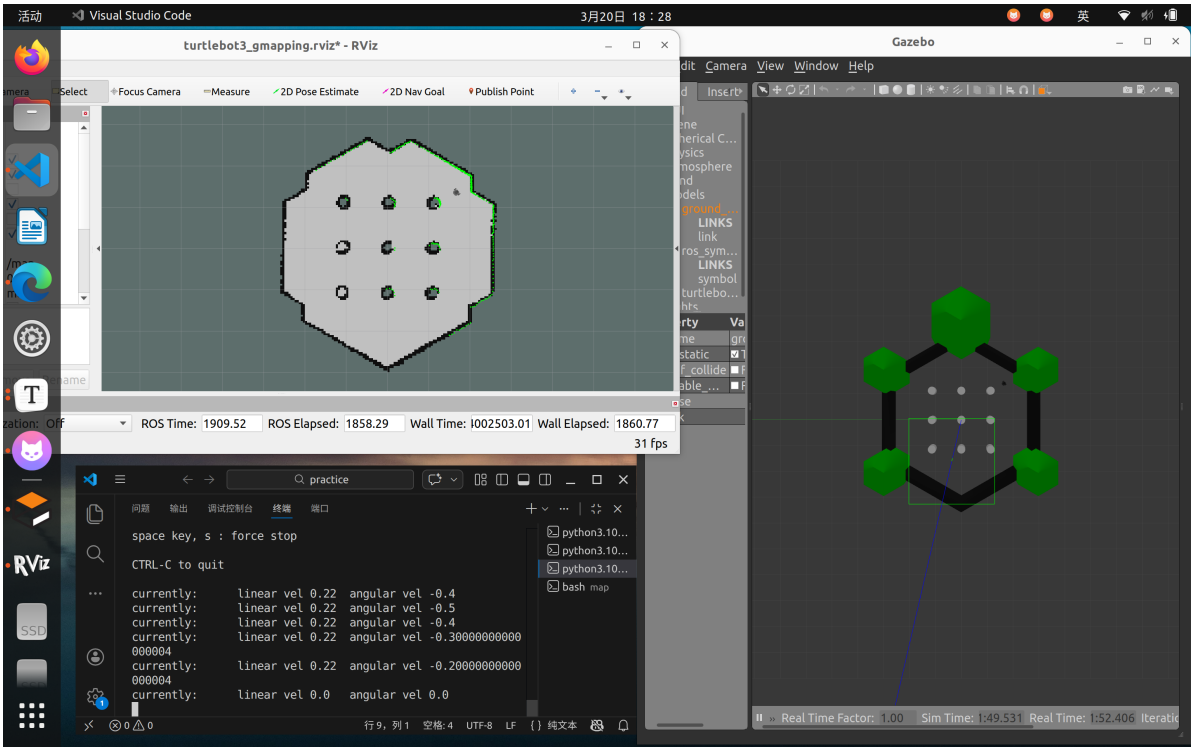
使用teleop键盘控制移动：`roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch`

最后保存地图：`rosrun map_server map_saver -f ~/map`

初始界面：

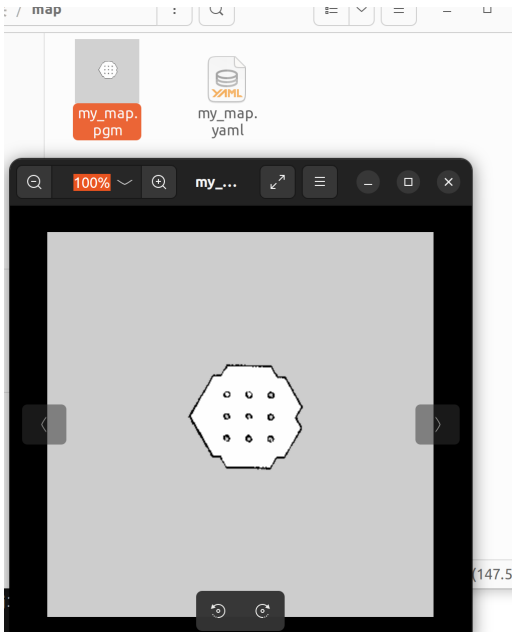


结束界面：



保存好的图：

`my_map.pgm`和`my_map.yaml`



自己建图+ turtlebot3：

在gazebo自行搭建，保存，名称为`my_world.world`

启动仿真环境`roslaunch my_sim my_sim.launch`

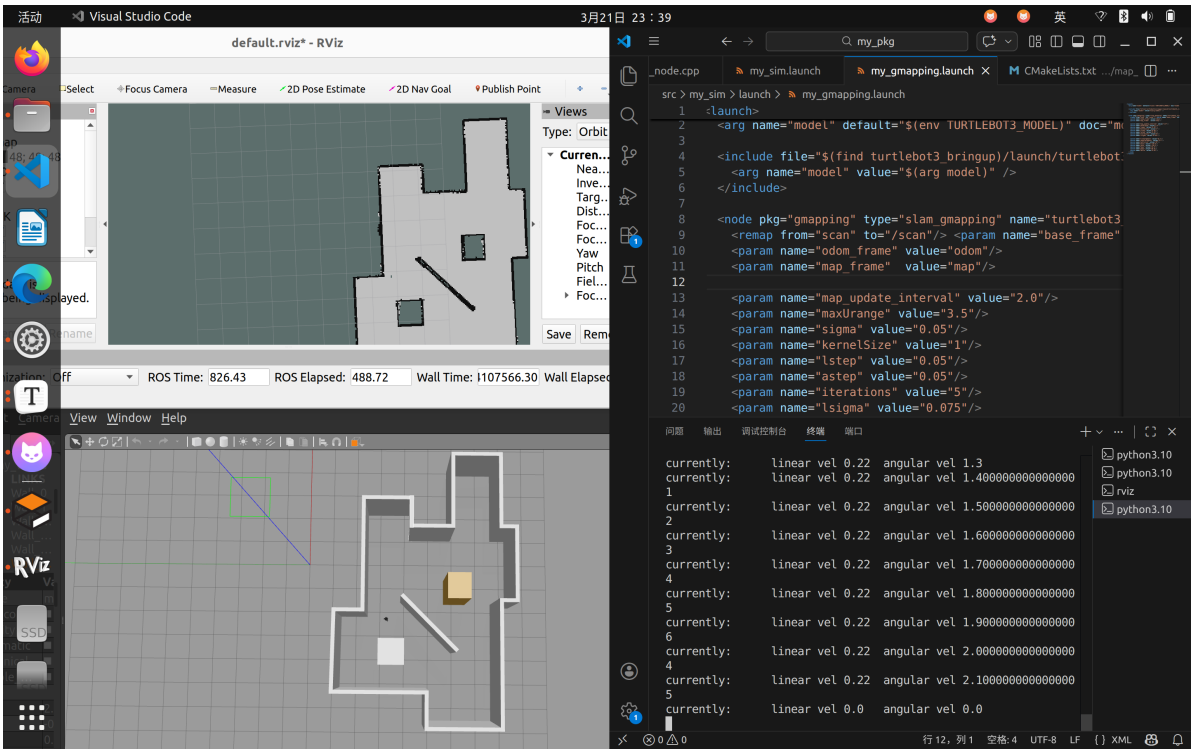
启动gmapping `roslaunch my_sim my_gmapping.launch`

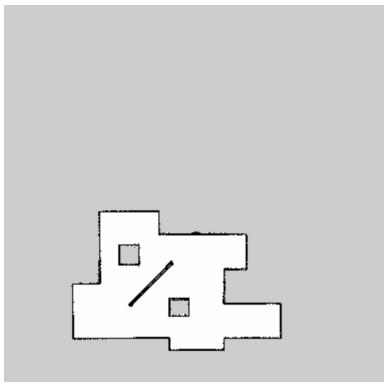
启动rviz `roslaunch turtlebot3_slam turtlebot3_slam_rviz.launch`

启动键盘遥控`roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch`

扫描完成以后，保存

`rosrun map_server map_saver -f ~/map/map`





自己建机器人+自己的图：

简单实现一个带激光雷达的机器人：`my_robot.urdf.xacro`

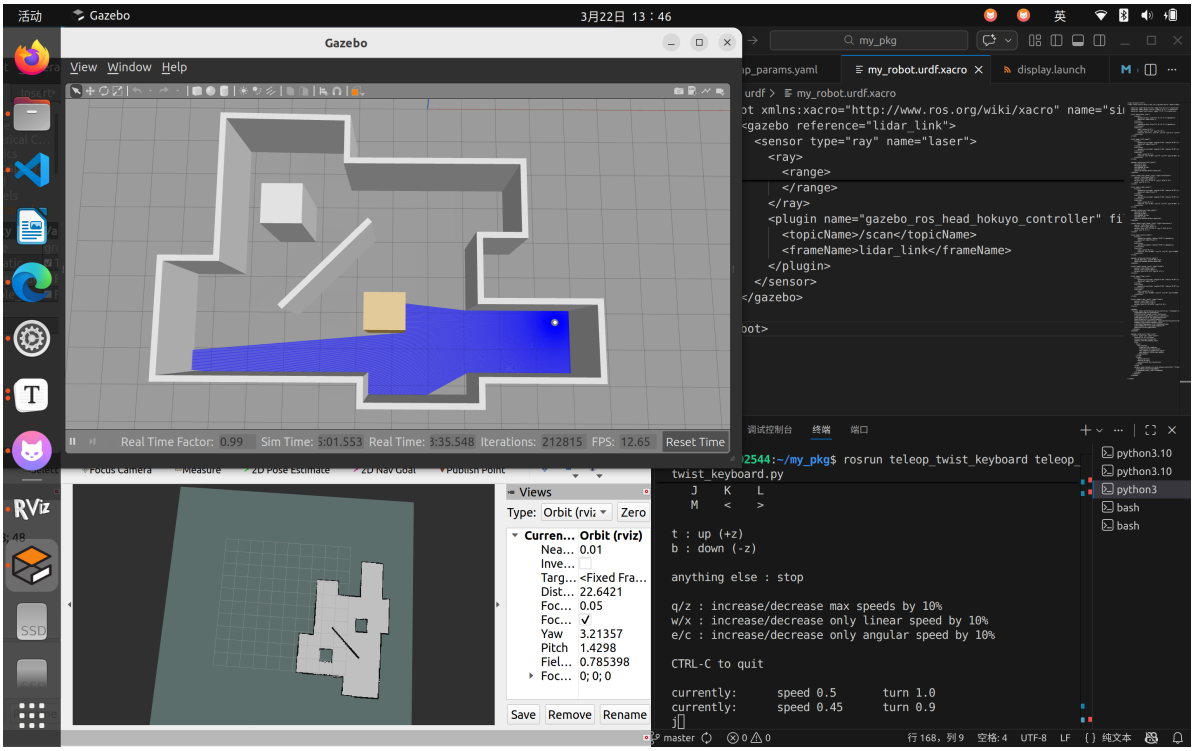
启动仿真：`roslaunch my_sim display.launch`

启动建图：`roslaunch my_sim dis_gmapping.launch`

启动键盘控制：`rosrun teleop_twist_keyboard teleop_twist_keyboard.py`

扫描完成以后，保存

`rosrun map_server map_saver -f ~/map/map`



问题：

为什么gmapping算法需要里程计信息?

答：里程计信息包括机器人的位姿，速度以及编码器数据；gmapping算法基于RBPF粒子滤波，该算法第一步是采样，通过获取里程计信息来确定粒子范围，从而提高效率；否则，不知道机器人的位置，只能假设可能出现在任何地方，那么就会**在全图范围内撒下粒子，效率大大降低；**

考核3 实现机器人自主定位和导航

1.航点导航

> 本质是向 `move_base` 节点发送一个类型为 `move_base_msgs/MoveBaseActionGoal` 的消息

基于turtlebot3 的航点导航

首先启动gazebo`roslaunch turtlebot3_gazebo turtlebot3_world.launch`

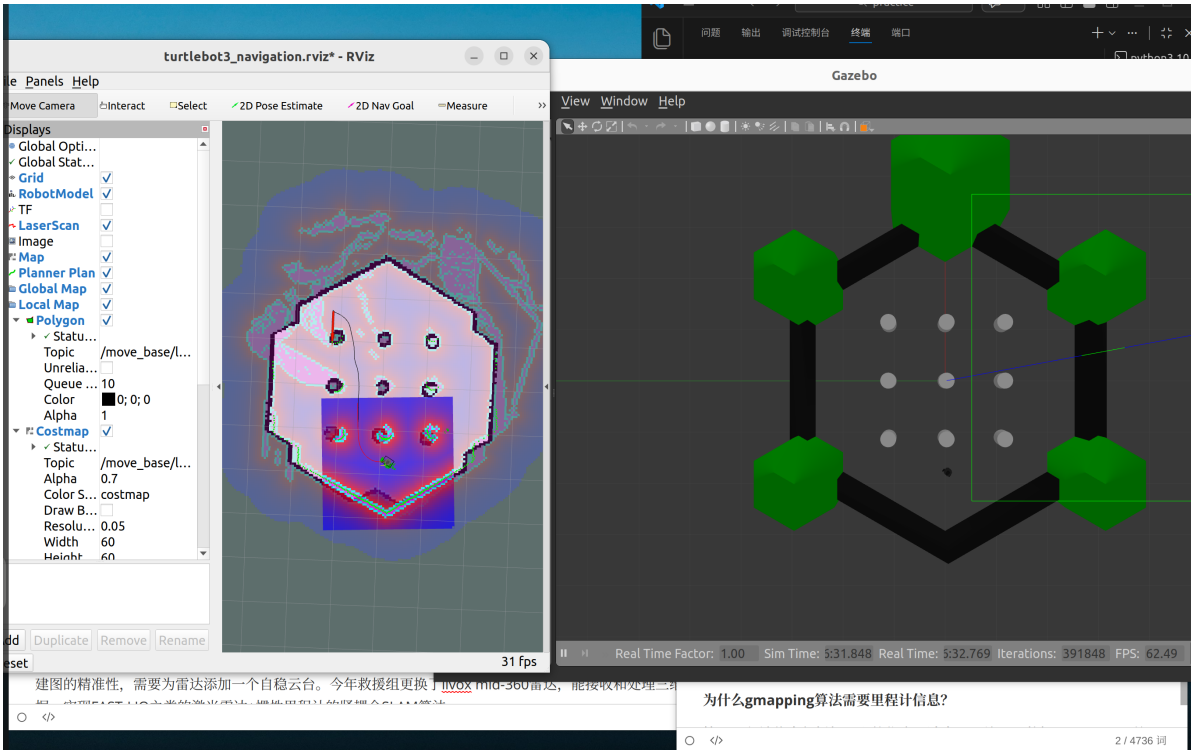
考核2获得的地图保存在~/map文件夹下

运行以下指令自动启动`move_base`并加载了配置文件

`roslaunch turtlebot3_navigation turtlebot3_navigation.launch map_file:=$HOME/map/my_map.yaml`

> **实现一：在rviz设置目标点**

先点击`2D Pose Estimate`同步初始机器人位置，然后点`2D Nav Goal`设置一个目标点，机器人自动规划路径如下图：



> **实现2：写一个`nav1.cpp`，调用`actionlib`库**

控制机器人移动有两种方式：发布`geometry_msgs/PoseStamped`话题（Topic）或者使用`Actionlib`；这里采用后者；

可执行文件名为nav1`rosrun my_sim nav1`

nav1.cpp如下（单点导航）

```
#include <ros/ros.h>
#include <move_base_msgs/MoveBaseAction.h>
#include <actionlib/client/simple_action_client.h>
typedef actionlib::SimpleActionClient<move_base_msgs::MoveBaseAction>
MoveBaseClient;
int main(int argc, char** argv) {
    ros::init(argc, argv, "nav1");
    MoveBaseClient ac("move_base", true);
    ac.waitForServer(ros::Duration(5.0));
    move_base_msgs::MoveBaseGoal goal;
    goal.target_pose.header.frame_id = "map";
    goal.target_pose.header.stamp = ros::Time::now();
    goal.target_pose.pose.position.x = 1.0;  // 目标 x 坐标
    goal.target_pose.pose.position.y = 2.0;  // 目标 y 坐标
    goal.target_pose.pose.orientation.w = 1.0;
    ac.sendGoal(goal);
    ac.waitForResult();
    return 0;
}
```

> 💭 **如果实现多航点导航——重新写一个导航类，用vector存储导航点，如果要删除使用erase函数**

```
#include <ros/ros.h>
#include <move_base_msgs/MoveBaseAction.h>
```

```cpp
#include <actionlib/client/simple_action_client.h>
#include <vector>

typedef actionlib::SimpleActionClient<move_base_msgs::MoveBaseAction>
MoveBaseClient;

class Navigator {
private:
    MoveBaseClient ac;
    std::vector<std::pair<double, double>> points;  // 存储(x, y)坐标
    int current_idx;

public:
    Navigator() : ac("move_base", true), current_idx(0) {
        ac.waitForServer(ros::Duration(5.0));
    }

    // 添加目标点
    void addPoint(double x, double y) {
        points.push_back({x, y});
    }

    // 创建目标
    move_base_msgs::MoveBaseGoal makeGoal(double x, double y) {
        move_base_msgs::MoveBaseGoal goal;
        goal.target_pose.header.frame_id = "map";
        goal.target_pose.header.stamp = ros::Time::now();
        goal.target_pose.pose.position.x = x;
        goal.target_pose.pose.position.y = y;
        goal.target_pose.pose.orientation.w = 1.0;
        return goal;
    }

    // 循环导航（可选）
    /*
    void runLoop() {
        while (ros::ok()) {
            for (auto& p : points) {
                ac.sendGoal(makeGoal(p.first, p.second));
                ac.waitForResult();

                if (ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED)
{
                    ros::Duration(1.0).sleep();
                } else {
                    ROS_WARN("导航失败");
                }
            }
            ROS_INFO("完成一轮，继续循环");
        }
    }
};
*/

int main(int argc, char** argv) {
    ros::init(argc, argv, "nav1");

    Navigator nav;
```

```
    nav.addPoint(0, -3.0);    // 原始目标点
    //在这里可以多添加点实现多点导航
    //nav.runLoop();

    return 0;
}
```

自定义机器人+自定义地图航点导航

- 第一步：配置Navigation stack

创建`config`文件夹，给机器人配置四个文件（见my_sim/config）

`costmap_common_params.yaml`：**机器人尺寸，膨胀半径，传感器配置**

`local_costmap_params.yaml`：**局部地图配置（动态更新、滚动窗口等）**

`global_costmap_params.yaml`：**全局地图配置（使用静态地图、坐标系等）**

`dwa_local_planner_params.yaml`：**DWA 局部规划器（速度限制、加速度等）**

- 第二步：编写launch文件，关联起所有部件

  启动move_base,关联刚刚写的文件

创建一个`display.launch`包含自定义地图和机器人的加载；还有一个`move_base.launch`，加载上一部配置的文件和amcl等；最终只启动`move_base.launch`即可

- 第三步：设定目标开始导航

  打开rviz设定目标点或者参考上面写航点导航的文件（这里直接用rviz设定目标了）✔️

🤔**遇到的问题：**

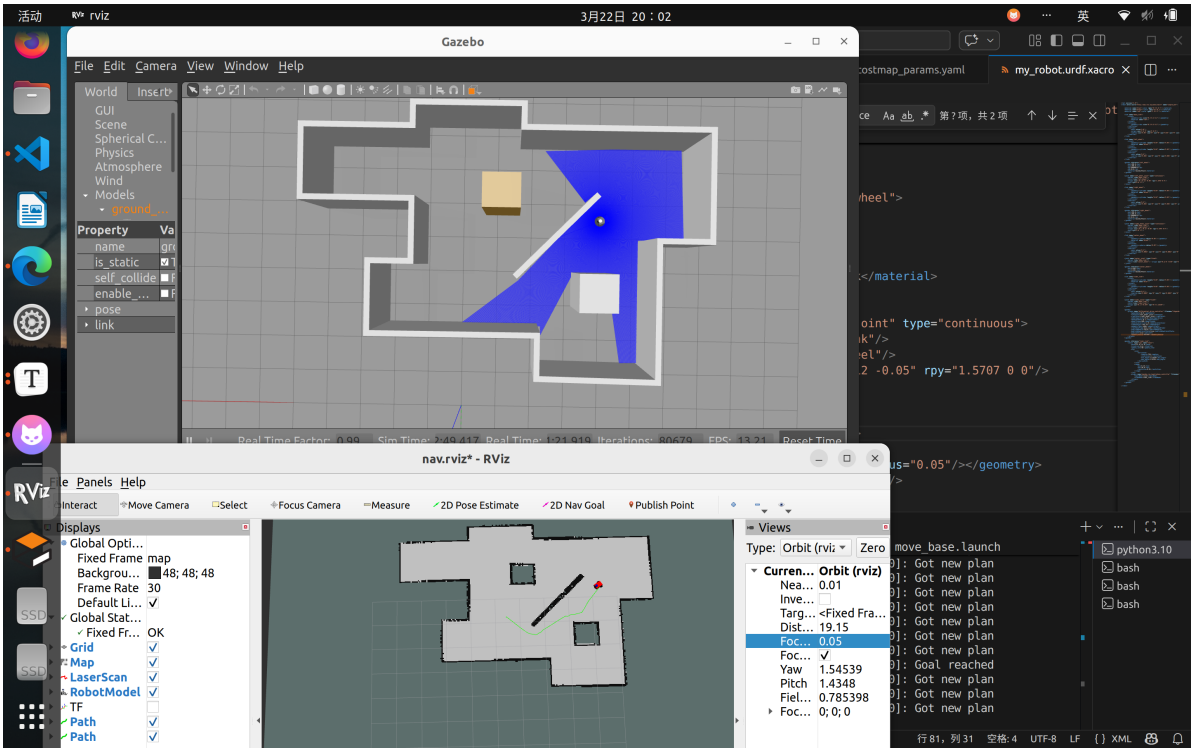
？机器人总是反向走：

尝试 换了插件里的 `leftJoint` 和 `rightJoint` 名字，不可行；最终发现电机旋转方向与 ROS 的 x轴正方向相反，修改 URDF 中轮子 `joint` 的 `<axis xyz="0 0 1"/>` 为 `0 0 -1`，同时将差速插件的 `odometrySource` 从 `world` 改为 `encoder`，确保 RViz 里的位姿变化完全由轮子转速决定，消除了 Gazebo 世界坐标系的干扰；（问AI）

？机器人总是撞墙或者是在窄通道里卡住不动了：

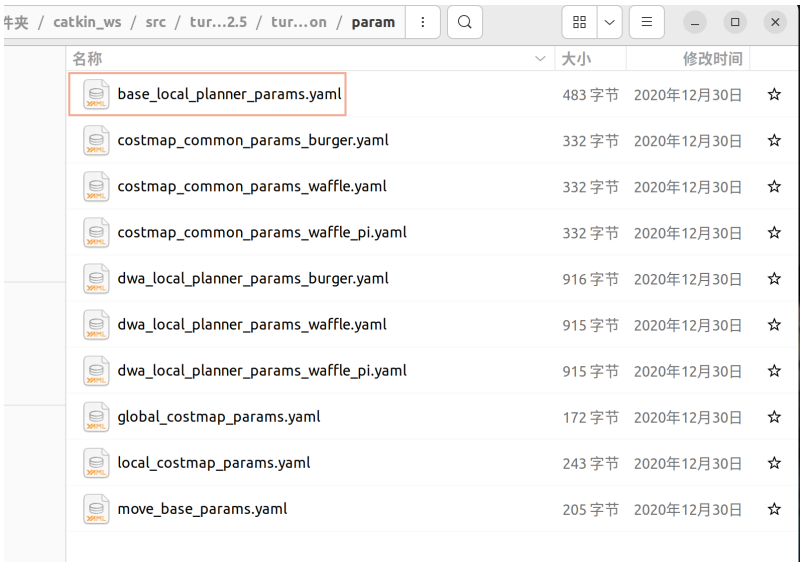
调低 `occdist_scale`（避障权重）；由于建的图比较小，机器人尺寸有一点大，设 `robot_radius` 不精确，窄通道过不去,于是在 `costmap_common_params.yaml` 中定义 `footprint`；不断调整`inflation_radius`

导航图：



move_base原理

`turtlebot3_navigation/param/` 目录下有以下配置文件，包括通用代价地图设置+局部地图范围+全局地图设置+dwa_local_planner_params+节点控制参数



打开每一个文件，可以看到如下配置：

- **base_local_planner_params**

  详细参数参考https://wiki.ros.org/base_local_planner

  - `max_vel_x / min_vel_x`: 小车前后移动的最大和最小速度（单位：m/s）
  - `max_vel_theta / min_vel_theta`: 最大旋转速度（单位：rad/s）
  - `min_in_place_vel_theta`: 机器人**原地自转**时的最小速度
  - `acc_lim_x / acc_lim_theta`: 线加速度和角加速度限制（漂移时调小）
  - `xy_goal_tolerance: 0.10`: 距离误差

- `yaw_goal_tolerance: 0.05`: 角度误差（如果小车到达目标点后一直**原地左右扭动**不停，说明你的角度容差设得太小）
- `holonomic_robot: false`: 差速轮设置（flase:非全向机器人，只能前后走+旋转；true:全向机器人）
- `sim_time: 0.8`: 预看时间（模拟未来 0.8 秒内的运动轨迹）、
- `vx_samples / vtheta_samples`: 采样数
- `sim_granularity: 0.05`: 步长

- costmap

  参数：https://wiki.ros.org/costmap_2d?distro=noetic#costmap_2d.2Flayered.Parameters

- dwa_local_planner

  https://wiki.ros.org/dwa_local_planner?distro=noetic#Overview

- move_base

  https://wiki.ros.org/move_base?distro=noetic#Parameters

在`turtlebot3_navigation/launch/move_base.launch`中可以看到调用逻辑：

move_base.launch

```
<launch>
  <!-- Arguments -->
  <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type [burger, waffle, waffle_pi]"/>
  <arg name="cmd_vel_topic" default="/cmd_vel" />
  <arg name="odom_topic" default="odom" />
  <arg name="move_forward_only" default="false"/>

  <!-- move_base -->
  <node pkg="move_base" type="move_base" respawn="false" name="move_base" output="screen">
    <param name="base_local_planner" value="dwa_local_planner/DWAPlannerROS" />
    <rosparam file="$(find turtlebot3_navigation)/param/costmap_common_params_$(arg model).yaml" command="load" ns="global_costmap" />
    <rosparam file="$(find turtlebot3_navigation)/param/costmap_common_params_$(arg model).yaml" command="load" ns="local_costmap" />
    <rosparam file="$(find turtlebot3_navigation)/param/local_costmap_params.yaml" command="load" />
    <rosparam file="$(find turtlebot3_navigation)/param/global_costmap_params.yaml" command="load" />
    <rosparam file="$(find turtlebot3_navigation)/param/move_base_params.yaml" command="load" />
    <rosparam file="$(find turtlebot3_navigation)/param/dwa_local_planner_params_$(arg model).yaml" command="load" />
    <remap from="cmd_vel" to="$(arg cmd_vel_topic)"/>
    <remap from="odom" to="$(arg odom_topic)"/>
    <param name="DWAPlannerROS/min_vel_x" value="0.0" if="$(arg move_forward_only)" />
  </node>
</launch>
```

2.自主探索算法

Frontier-based Exploration

> 占据栅格地图中，每个栅格有三种状态：Free,Occupied,Unknown
>
> frontier：机器人可以安全到达、并且紧挨着还没探索过的灰色地带的那一圈边界点

- **算法流程：**
  - **边界提取：**

    算法通过监听`nav_msgs/OccupancyGrid`话题获取全局代价地图（Global Costmap）

    查找：对每一个标记成探索过的点（free）检查8领域，若一个 Free 状态的点邻域内至少存在一个Unknown的点，且该点本身未被标记为障碍物，则将其定义为 **边界点 (Frontier Point)**

    连通分量标记：利用 **BFS** 或 **DBSCAN 聚类算法**，将空间位置相邻的边界点组合成一个个独立的**边界簇 (Frontier Clusters)**

过滤：统计集合中点的数量，如果数量太少就剔除；计算算簇的几何包围盒或直径（欧几里得长度），如果小于机器人的包围圆直径（或安全通行宽度）就剔除；

质心重定位：先计算质心，避免落在unknown或者障碍物区域，需要在点集中找一个点到质心的欧几里得距离最小作为最终法target goal；

- **目标点评估选取**

  上一步提取出多个集合（边界簇），需要评估选择一个最优目标去探索，也就是选出target goal；

  通过 **效用函数** 选出最优目标：

  信息增益：评估该边界簇的大小（点数越多通常意味着背后隐藏的未知空间越大）；

  路径代价：通过全局规划器（如 Dijkstra 或 A*）计算当前位姿到边界簇质心的实际路径代价；

  效用函数权衡：采用加权评分机制

- **路径规划**

  上一步确定了目标，接下来到了ROS Navigation Stack 开始工作，进行规划路线；局部路径规划器（DWA 或 TEB）负责处理动态障碍物并执行具体的电机指令；移动过程中，算法持续更新地图

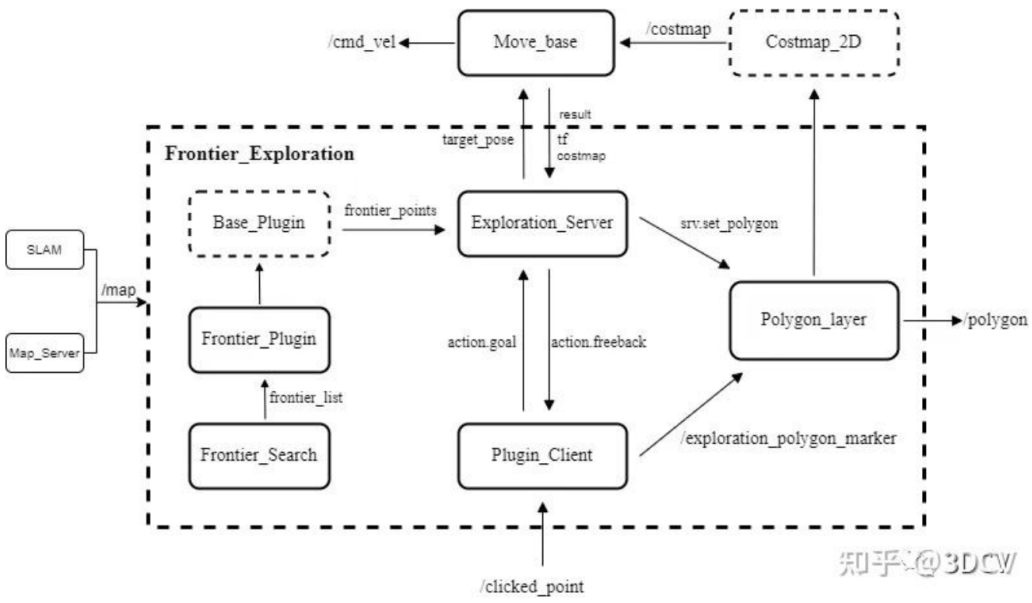
- **闭环更新与终止判定**

  更新边界点的状态，大的边界可能变成小块；当边界点集合为空集后停止并触发 `map_server` 进行持久化存储

三种软件包实现

1. **frontier_exploration**

> 基于**插件机制**，作为 `move_base` 的一部分；传统**遍历式**实现 ✍️



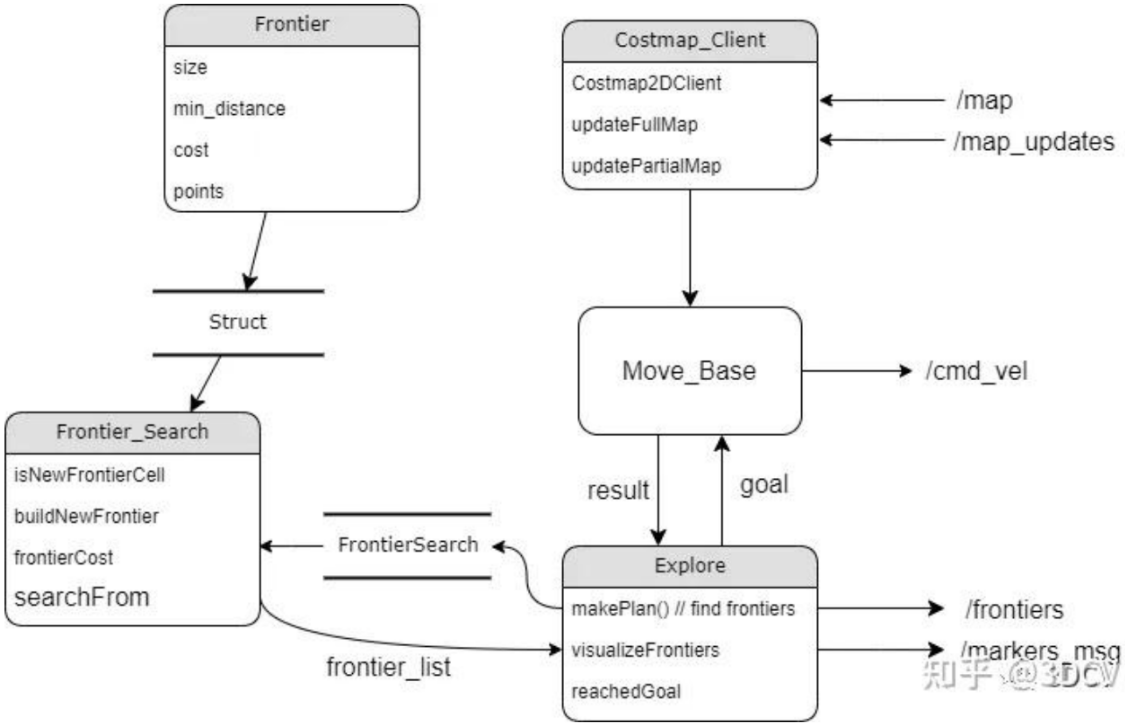
- SLAM建图，输入环境的二维栅格地图
- **Frontier_Search** 模块：基于 `/map` 数据，遍历地图识别“已知区域与未知区域的边界点”，生成 `frontier_list`（前沿点列表）
- **Frontier_Plugin** 模块：接收 `frontier_list`，进行聚类、筛选与优化（如去除过小前沿、合并邻近点），输出标准化的 `frontier_points`

- **Exploration_Server**：接收 `frontier_points`，结合代价地图 `/costmap`、坐标变换 `/tf` 信息，选择最优探索目标点，生成 `target_pose`，将目标点封装为`actionlib`的goal，发送给`Move_base`执行；提供服务接口 `srv.set_polygon`，用于设置探索区域的多边形边界
- **Plugin_Client** 模块：发布 `/exploration_polygon_marker`，在 RViz 中可视化探索区域多边形；接收 `/clicked_point`（用户点击点），支持手动指定探索区域
- **Polygon_layer** 模块：接收 `Exploration_Server` 的 `srv.set_polygon` 服务调用，设置探索边界多边形；向 `Costmap_2D` 提供区域约束信息，限制探索与导航范围；发布 `/polygon` 话题，向外广播当前探索区域的多边形数据

---

2. **explore_lite**

> **独立节点**，不侵入导航栈；传统**遍历式实现**



- **Frontier** 结构体：封装了单个前沿区域的核心属性，用于后续的筛选与决策

  `size`：前沿区域的栅格数量 / 面积

  `min_distance`：到机器人当前位置的最小距离

  `cost`：到达该前沿的代价（如路径成本、信息增益权重）

  `points`：构成该前沿区域的所有栅格点集合

- **Frontier_Search** 模块：

  `isNewFrontierCell()`：判断单个栅格是否为 “已知 - 未知” 边界的新前沿单元格。

  `buildNewFrontier()`：将邻近的前沿单元格聚类，生成完整的 `Frontier` 结构体实例。

  `frontierCost()`：计算每个前沿区域的综合代价（结合距离、信息增益等策略）。

`searchFrom()`：从机器人当前位姿出发，遍历地图生成所有前沿区域，输出 `frontier_list`（前沿区域列表）。

3. rrt_exploration

> 用**RRT 算法替代传统栅格遍历**做前沿检测和路径规划；随机**采样式实现**

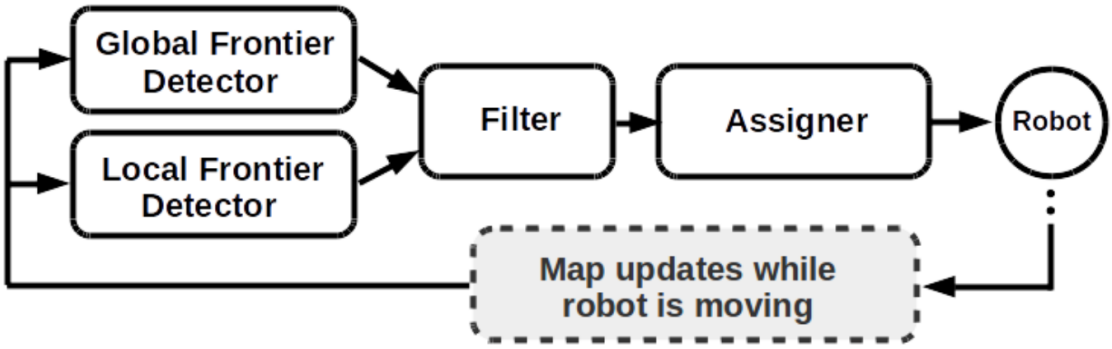
- RRT（快速探索随机树）算法：

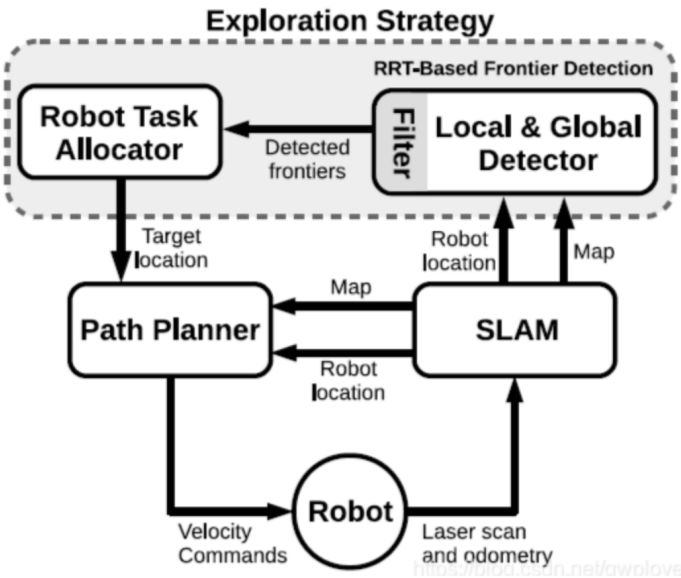
  核心思想是**通过在空间中随机采样点并逐步构建一棵树形结构（搜索树），来快速探索空间并找到从起点到终点的可行路径**

  首先算法通过在**状态空间中随机采样**来探索环境（二维平面中即为整个地图范围），以概率方式逐步探索空间；采样策略有多种，比如均匀随机采样适用于完全未知的环境，目标偏置采样能以一定概率直接选择目标点从而加速收敛，而前沿导向采样则优先在已知与未知区域的边界采样，特别适合自主探索任务；

  其次将**新节点连接到最近的树节点**：每次随机采样后，算法会在已有树中寻找距离采样点最近的节点，然后沿采样点方向生长固定步长得到新节点；

  通过**逐步生长**的方式，树结构**覆盖整个空间**；采样点在整个状态空间中均匀分布，面积更大的未探索区域有更高的概率被采中





- **空间采样与边界捕获**：不是遍历所有地图（提高效率了），而是在工作空间**内生成随机采样点，**只有当采样点落在`Free`且与父节点的连线不穿过障碍物时，RRT 树才会扩展；终点落在`Unknown`时，该线段与未知区域的交点被实时捕获为一个**候选边界点**
- **Global Frontier Detector（全局前沿检测器）**

  在全局地图范围内采样，先随机生成一个坐标点p,在已有的RRT树中找到距离最近的节点q，从q-->p延伸固定一个步长得到新点，判断是否符合条件，生成全局前沿列表`global_frontier_list`

  `rrtGlobalSearch()`（全局 RRT 采样）`extractGlobalFrontiers()`（提取全局前沿）
- **Local Frontier Detector（局部前沿检测器）**

  逻辑同上，但是采样范围被限制在机器人当前位置的一个小半径圆周内，快速检测局部未知区域边界，生成局部前沿列表`local_frontier_list`
- **Filter & Assigner（过滤器与分配器）**
  - Filter：收集上一步得到的边界点进行聚类（**Mean-Shift（均值漂移）**：找到点的密度峰值作为中心），对于每一个聚类后的中心点，计算其周围半径 R 内 Unknown 栅格的数量
  - Assigner：决定target

    计算机器人到每个候选边界点的欧几里得距离或路径长度，如果是单台机器人，使用贪婪策略（遍历所有边界点，选出得分最高的那个点作为`move_base`的目标）；多台机器人：使用匈牙利算法（使得所有机器人到达各自目标的总代价最小）；最后发给`move_base`。
- **RRT Path Planner（RRT 路径规划器）**：
  - 输入：`target_pose`、`/robot_pose`、代价地图
  - 核心逻辑：再次调用 RRT 算法，在代价地图中搜索从机器人当前位姿到目标前沿的无碰撞路径，输出`global_path`（全局路径）

对比：AI生成

维度	frontier_exploration（插件化架构版）	explore_lite（轻量化基础版）	rrt_exploration（RRT 采样驱动版）
核心算法原理	基于**栅格遍历**的前沿检测：1. 遍历地图识别边界点2. 插件化扩展不同前沿筛选策略	基于**栅格遍历**的前沿检测：1. 无插件化，直接聚类生成前沿2. 轻量级代价函数筛选最优前沿	基于**RRT 采样**的前沿检测：1. 通过 RRT 随机采样搜索未知区域边界2. 同步用 RRT 完成路径规划
架构设计	工程化插件架构：- 核心：Base_Plugin + Frontier_Plugin- 分工：Exploration_Server（服务端）+ Plugin_Client（客户端）- 支持区域约束（Polygon_layer）	极简轻量化架构：- 核心：Frontier_Search + Explore- 直接对接 Move_Base，无分层设计- 无独立客户端 / 服务端	采样驱动全栈架构：- 核心：Global/Local RRT 检测器 + Task Allocator- 集成 SLAM、规划、执行全链路- 支持多机器人任务分配
前沿检测逻辑	两步式处理：1. Frontier_Search 生成原始前沿列表2. Frontier_Plugin 做聚类 / 筛选 / 优化	一站式处理：1. Frontier_Search 内置聚类 + 代价计算2. 直接输出可用前沿列表	双检测器模式：1. 全局检测器：全地图 RRT 采样找大范围前沿2. 局部检测器：机器人周边轻量化采样，保证实时性
路径规划依赖	完全依赖 Move_Base（A*/Dijkstra 算法）	完全依赖 Move_Base（A*/Dijkstra 算法）	自研 RRT 路径规划器：与前沿检测共用 RRT 框架，不依赖 Move_Base
多机器人支持	无，仅适配单机器人	无，仅适配单机器人	原生支持：通过 Task Allocator/Assigner 实现多机负载均衡分配
工程特性	1. 支持多边形探索区域约束2. 提供 Action/Service 接口3. 可视化探索区域（polygon_marker）	1. 仅支持基础代价约束2. 仅提供话题接口3. 可视化前沿区域（markers_msg）	1. 无专门区域约束模块2. 算法层接口，无标准化工程接口3. 聚焦核心逻辑，轻量化可视化
核心优势	1. 插件化易扩展，可替换前沿算法2. 工程化程度高，适配工业级场景3. 区域约束能力强	1. 代码量少，易部署调试2. 栅格遍历稳定，适配小场景3. 学习成本低，新手友好	1. RRT 适配非结构化 / 复杂环境（迷宫、废墟）2. 多机器人支持完善3. 大地图探索效率高于栅格法
核心劣势	1. 架构复杂，调试成本高2. 栅格遍历在大地图效率低3. 仅支持单机器人	1. 功能单一，无扩展能力2. 复杂环境路径规划差3. 无区域约束 / 多机能力	1. RRT 采样有随机性，结果不稳定2. 工程化程度低，部署成本高3. 实时性略低于栅格法

维度	frontier_exploration（插件化架构版）	explore_lite（轻量化基础版）	rrt_exploration（RRT采样驱动版）
适用场景	1. 室内结构化场景（办公室、展厅）2. 工业级单机器人项目3. 需要灵活扩展算法的场景	1. 室内小场景验证（实验室、小房间）2. 入门级算法学习3. 快速原型验证	1. 室外 / 复杂室内场景（废墟、迷宫）2. 多机器人协同探索3. 非结构化环境探索研究

基于turtlebot3实现explore_lite自主建图

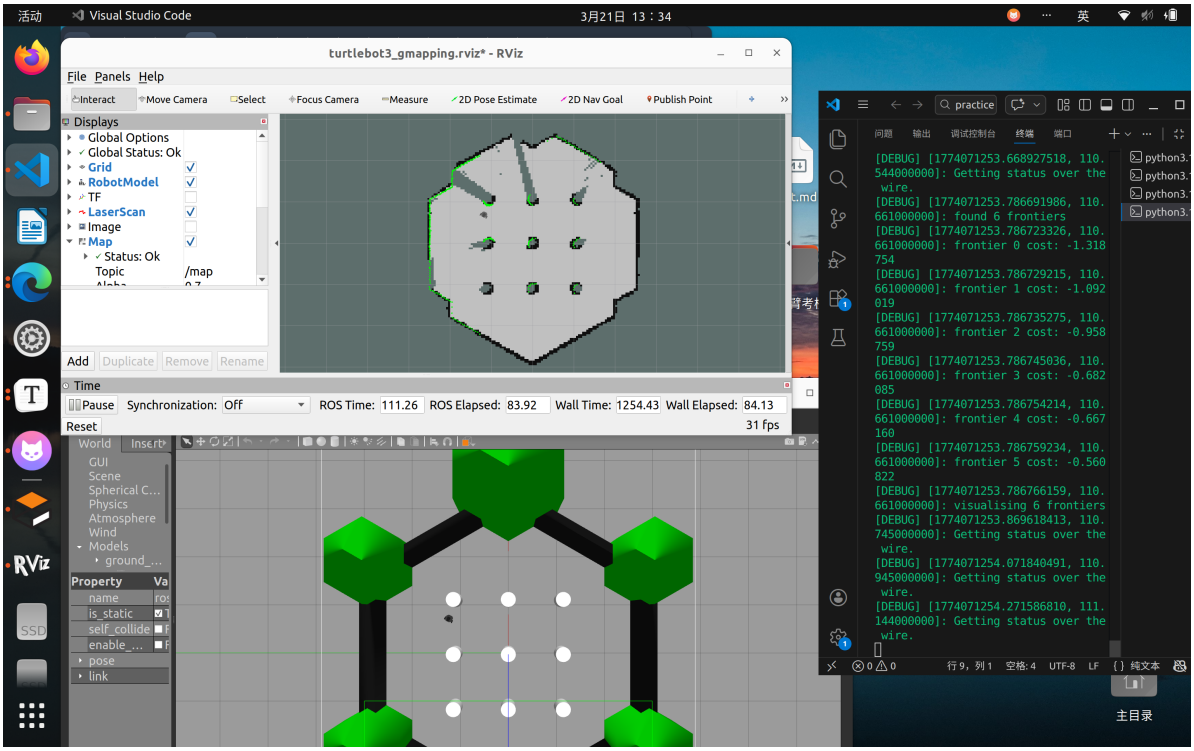
启动gazobo `roslaunch turtlebot3_gazebo turtlebot3_world.launch`

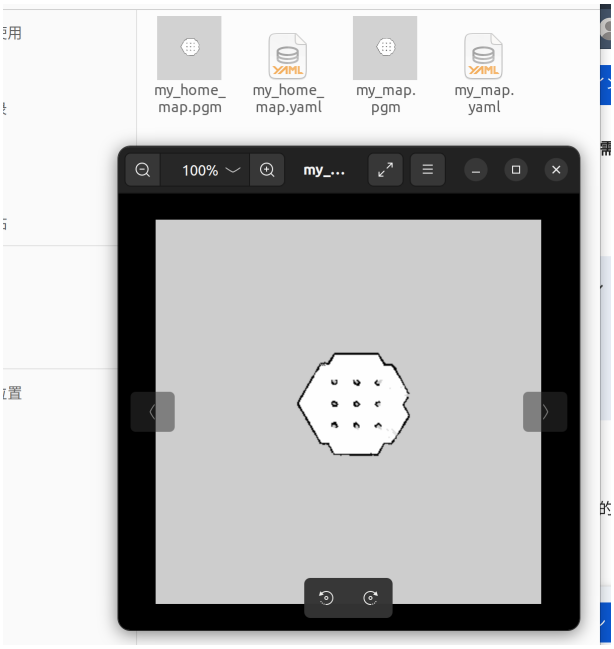
启动gmapping `roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping`

启动move_base `roslaunch turtlebot3_navigation move_base.launch`

启动自主探索算法 `roslaunch explore_lite explore.launch`

保存图：`rosrun map_server map_saver -f ~/my_home_map`





自定义机器人实现explore_lite自主建图

- 第一步：同航点导航，创建配置4个文件

1. `costmap_common_params.yaml`
2. `local_costmap_params.yaml`
3. `global_costmap_params.yaml`
4. `dwa_local_planner_params.yaml`

- 第二步：编写一个launch ,需要实现以下功能： （/my_sim/launch/auto_explore.launch）

打开gazebo，加载my_world.world

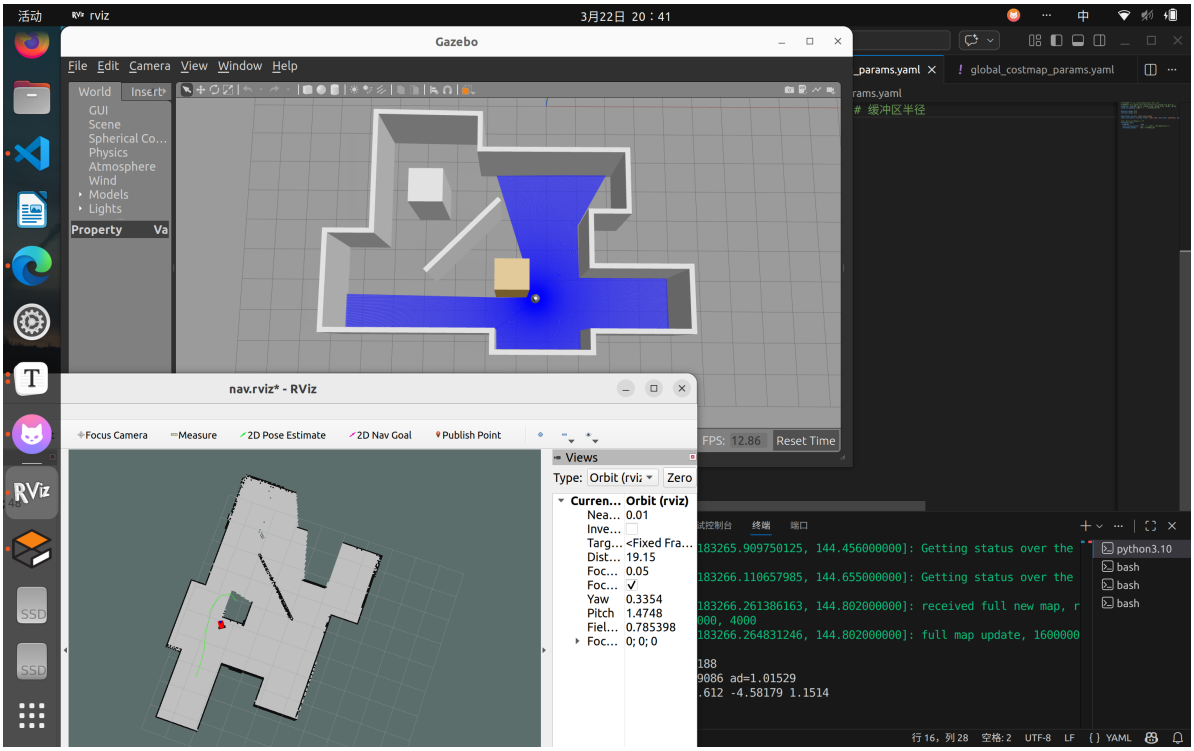
启动gmapping，监听 `/scan`，发布 `/map` 话题

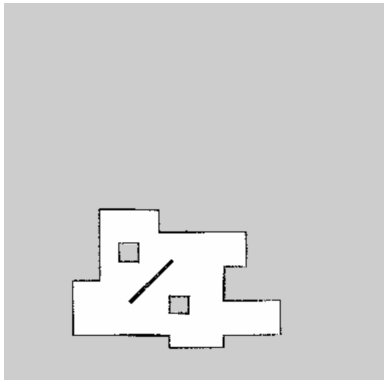
组合集结第一步的文件

启动explore_lite

- 第三步：保存

建图过程及结果如下





考核四：实现三维建图

1.FAST-LIO

原理

> 紧耦合的迭代扩展卡尔曼滤波将LiDAR特征点与IMU数据融合

1. **传统的三维建图及不断更新历程：**
    - 通过**激光雷达获得点云，将点云重合拼凑得到地图**；**ICP算法**：通过旋转和平移，使得两帧点云中的点距离最近时拼凑，存在速度太慢（每一帧有几万个点），易丢失数据（机器人转太快，两帧之间重叠少），畸变（扫描过程中机器人在动）等问题；
    - **LOAM**：不是提取所有的点，而是只抓重点，即**特征点匹配（线/边缘和面）从而减少计算量**；**分离前后端，高频前端里程计 + 低频后端建图**；使用匀速模型（假设机器人在两帧之间既不加速也不减速）预估运动缓解畸变；
    - **LIO**：仅仅靠激光雷达，在高速运动或者特征缺失时会失效，此时引入IMU

        - **IMU（惯性单元）**：提供角速度和线加速度，采用**预积分**，在两帧点云之间的IMU数据合成相对位姿变换，相当于在下一帧点云还未扫描完成前**预测**机器人的位置；
    - **松耦合**：LiDAR 模块独立算出位姿 1，IMU 模块独立算出位姿 2,用**卡尔曼滤波或加权平均值**把两个位姿凑在一起（问题：当LiDAR算出 的位姿是错误的，整体出错）
    - **紧耦合**：把 LiDAR 的**每一个特征点**都视为一个观测方程而非计算独立位姿，在特征少的地方也能确定更新位姿；（修正：通过不断减小预测位姿下的特征点与全局地图中特征的**残差**来逼近真值）
    - **后端优化与全局一致**：回环检测 ——如果回到走过的地方，触发回环约束，将当前的点云和历史经过这里的点云进行ICP匹配

2. **FAST-LIO解决的问题和原理：**

- 背景问题：SLAM使用固态激光雷达时，由于视角小，获得的结果更容易退化（无特征环境或环境杂乱无序，无法拟合几何特征）；并且高分辨率的点云和IMU融合算力不足；同时当运用在无人机和发动机等器件时，其持续旋转给IMU带来噪声；
- 解决：
    - **紧耦合的迭代卡尔曼滤波**：适用在快速运动、噪声或杂乱的环境中，融合LiDAR特征点和IMU
    - 卡尔曼增益计算公式：降低LiDAR特征点的计算量

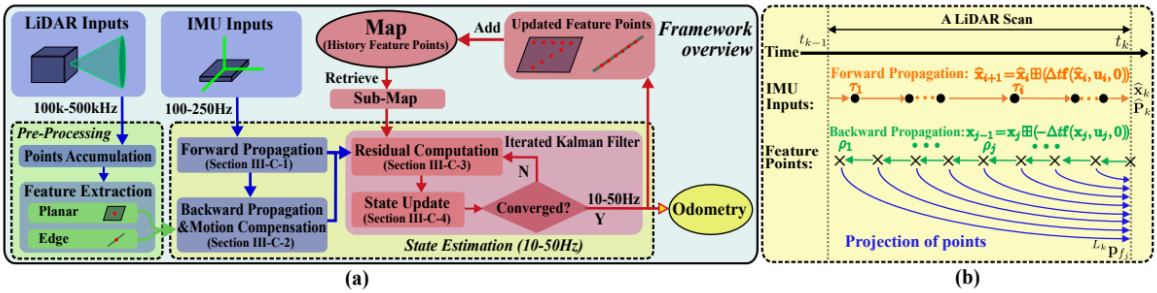


Fig. 2. System overview of FAST-LIO. (a): the overall pipeline; (b): the forward and backward propagation.

- **原理分析：**
    - **流程**：🏷️

      激光雷达的点云去畸变，送入**特征提取模块**，以获得平面和边缘特征；

      然后将提取的特征和IMU预测反馈到**状态估计模块**中，以进行10 Hz–50 Hz的状态估计，给出机器人下一时刻的位姿，同时产生一个协方差矩阵（衡量不确定性，越大即不确定性越大）；（本质是修正位姿）

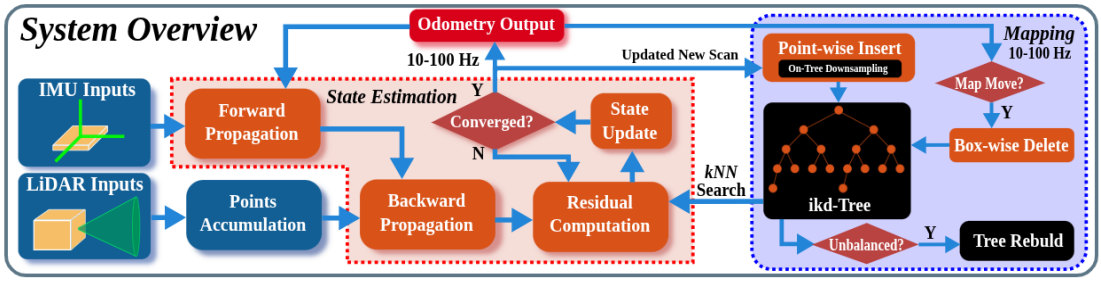
      当新特征点提取完成后，进入 **IEKF 的迭代循环**：把获得的平面和边缘特征投射到k-d tree维护的全局地图，计算**残差**；利用卡尔曼增益，不断微调位置和角度的偏差量，直到点云与地图完美贴合，残差减小到几乎为零；

      IEKF 的迭代收敛后，得到当前最精准的位姿估计，把结果传送给导航模块，并把这一帧处理好的特征点转换坐标系到全局，合并到 **k-d tree** 地图；

      最终使用更新后的地图继续匹配下一个点；
    - **同步与去畸变**： 利用**高频 IMU** 记录的角速度把这一帧内每一个激光点投影到该帧起始时刻的坐标系下，**消除运动畸变；**利用 IMU 的加速度和角速度进行惯性导航解算（动力学积分）**预估**出机器人在这一时刻的位姿和速度；同时计算协方差，计算当前位姿的不确定性；

- **空间约束建立**：将预测的位姿**投影到 k-d-Tree** 中，为当前帧的每一个特征点寻找最近的 5 个邻域点，利用这 5 个点通过最小二乘法拟合出一个小平面，计算特征点到这个拟合平面的垂直距离，这个距离就是 IMU 预测值与地图真实结构之间的**残差**；
- **迭代卡尔曼滤波更新 (IEKF)**：构建一个状态向量，包含**位姿**， **IMU 零偏 (Bias)** 和**重力矢量**；在修正旋转矩阵时，不使用加法，而是在**流形**上进行乘法更新，保证旋转矩阵永远满足正交性，避免数学计算发散；根据 IMU 协方差和激光点的残差，算出卡尔曼增益 K，如果激光点对得很准，就多修一点位姿；如果激光点在长走廊里找不到约束，就使用 IMU 的预测；重复 3-5 次，直到位姿不再变
    - 零偏：当传感器处于绝对静止状态时，输出值却不为零的**偏差**
    - 流形：弯曲的空间，让旋转的更新在数学上永远合法，防止旋转矩阵（必须满足R^T R = I，计算机中运行时浮点数不能精确存储）失真
- **动态地图维护**：当位姿完全确定后，把当前帧的点云插进地图；点云进入地图之前，系统会预先设定一个固定大小的栅格，在每个格子内只保留一个点，如果某个地方点太密，在进入地图前丢弃；

3. **FAST_LIO2的优点及原理**



- **较一代的升级点：** 📶
    - 无需特征提取，激光雷达扫描的原始点云直接
    - 新增ikd-tree，可以动态插入删除
- **原理分析：**
    - 流程：

      激光雷达数据输入，去畸变，和IMU的预测反馈到**状态估计模块**，进行 10 Hz–50 Hz 的高频状态估计；此时系统给出机器人下一时刻的位姿预判，并同步产生一个**协方差矩阵**，用以实时衡量当前位姿估计的不确定性；

      进入**IEKF迭代循环**：利用预测位姿将原始点云投射到由 **ikd-Tree** 维护的全局点云地图中，并快速搜索邻近点以计算**点到平面的残差**；随后利用卡尔曼增益不断微调误差状态，通过多次迭代使原始点云与复杂的地图环境完美贴合，直至残差收敛至极小值；

      当 IEKF 迭代收敛并得到当前最精准的位姿后，结果实时传送给导航模块；同时，这一帧原始点云被转换至全局坐标系，并触发 **ikd-Tree 的增量更新**：系统在后台自动监测空间密度，丢弃过密点并实时重新平衡树结构，将新点直接合并进全局地图；

      系统利用这棵**动态更新**的 ikd-Tree 全局地图，继续高效匹配下一帧进入的原始点云；
    - **kNN 搜索**：在给定的数据集里，搜索与目标点距离最近的 k 个邻居点的算法
    - **ikd-Tree**

      优点：支持高效地动态增删；能够始终保持tree的平衡；自带体素降采样功能

      保持树平衡：阿尔法-balanced 准则（可容忍的最大不平衡程度）+阿尔法-deleted 准则（可容忍的最大的被标记但未被删除的节点数）；再平衡算法，本质是把所有节点打乱，对剩下的点进行常规kdtree的构建

删除节点box-wise delete

插入节点与同步体素降采样Point Insertion with On-tree Downsampling

复现过程使用的包

1. livox_ros_driver2

   内部主参数和激光雷达配置说明，详细见https://github.com/Livox-SDK/livox_ros_driver2

参数	详细描述	默认
publish_freq	设置点云发布 频率浮点数据类型，推荐值为5.0、10.0、20.0、50.0等。最大发布频率为100.0 Hz。	10.0
multi_topic	如果LiDAR设备有一个独立主题来发布点云数据 0——所有LiDAR设备都使用相同的主题发布点云数据 1——每个激光雷达设备都有自己的主题来发布点云数据	0
xfer_format	设置点云格式 0 -- Livox pointcloud2（PointXYZRTLT） 点云格式 1 -- Livox 自定义点云格式 2 -- 标准 pointcloud2 （pcl：： PointXYZI） 点云格式，位于 PCL 库中（仅用于 ROS）	0

   自定义数据包，需要设置为1

雷达配置说明

对于mid-360雷达，打开MID360_config.json，需要配置以下内容：

`"ip" : "192.168.1.12"`:改为雷达的实际 IP 地址

`"host_net_info"`中的ip：改为运行驱动程序的电脑的网卡 IP

`"extrinsic_parameter"`：雷达在机器人的安装位置（如果有多个雷达，需要测量它们相对于机器人Base Link的偏移）

2.复现

安装PCL和Eigen

安装Livox-SDK2和livox_ros_driver2（连接 Livox激光雷达硬件与 ROS环境）

克隆并编译

```
git clone https://github.com/hku-mars/FAST_LIO.git
cd FAST_LIO
git submodule update --init
cd ../..
catkin_make
source devel/setup.bash
```

给包重命名为`test.bag`，查看包的信息：

`rosbag info test.bag`

```
wly@w2025302544:~$ rosbag info test.bag
path:        test.bag
version:     2.0
duration:    2:52s (172s)
start:       Jan 21 2026 19:18:28.34 (1768994308.34)
end:         Jan 21 2026 19:21:21.16 (1768994481.16)
size:        638.7 MB
messages:    36260
compression: none [576/576 chunks]
types:       livox_ros_driver2/CustomMsg [e4d6829bdfe657cb6c21a746c86b21a6]
             rosgraph_msgs/Log           [acffd30cd6b6de30f120938c17c593fb]
             sensor_msgs/Imu             [6a62c6daae103f4ff57a132d6f95cec2]
topics:      /livox/imu     34521 msgs    : sensor_msgs/Imu
             /livox/lidar    1727 msgs    : livox_ros_driver2/CustomMsg
             /rosout           12 msgs    : rosgraph_msgs/Log
```

可以看到数据包基于livox_ros_driver2的自定义消息

后续编译过程报错，查询发现FAST-LIO 源码默认适配livox_ros_driver一代，询问AI得到解决方法：

修改 `package.xml` 和 `CMakeLists.txt`，将所有 `livox_ros_driver` 依赖改为 `livox_ros_driver2`；

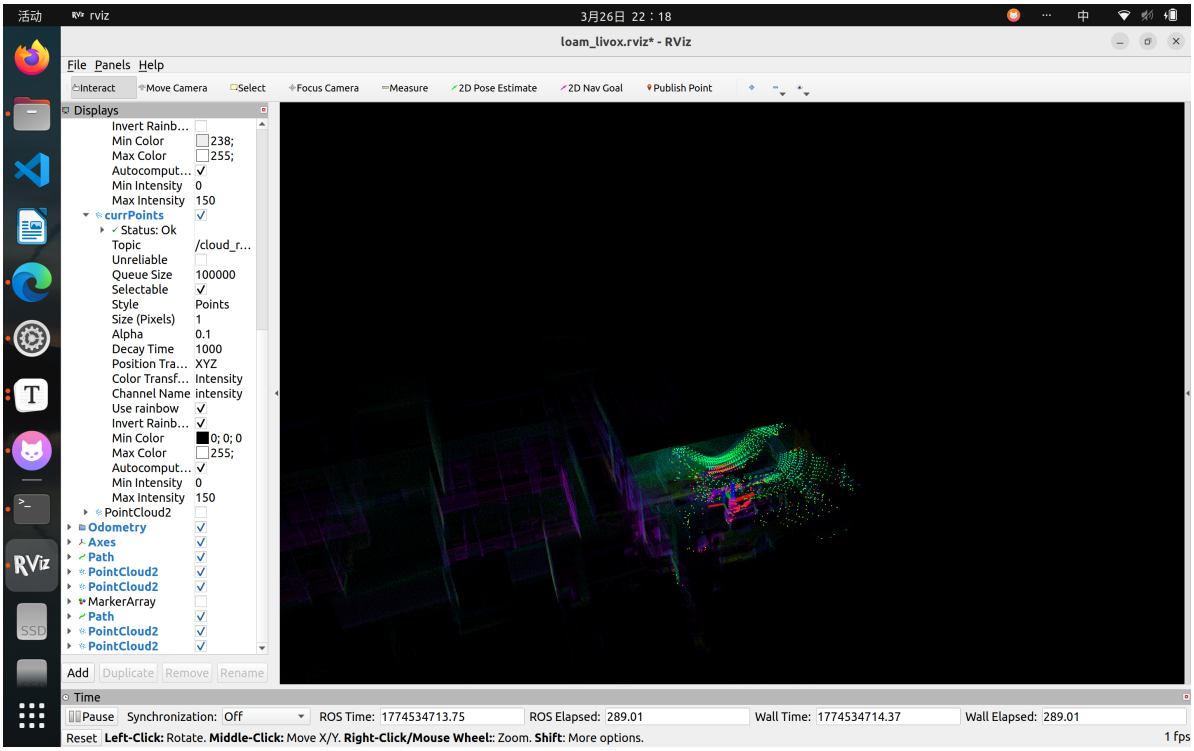
将源代码（`.cpp`, `.h`, `.hpp`）中的头文件引用和命名空间从 `livox_ros_driver::CustomMsg` 替换为 `livox_ros_driver2::CustomMsg`

在 `CMakeLists.txt` 中添加 `add_dependencies`，强制要求先生成二代驱动的消息头文件，再编译算法节点

编译运行

`roslaunch fast_lio mapping_mid360.launch`

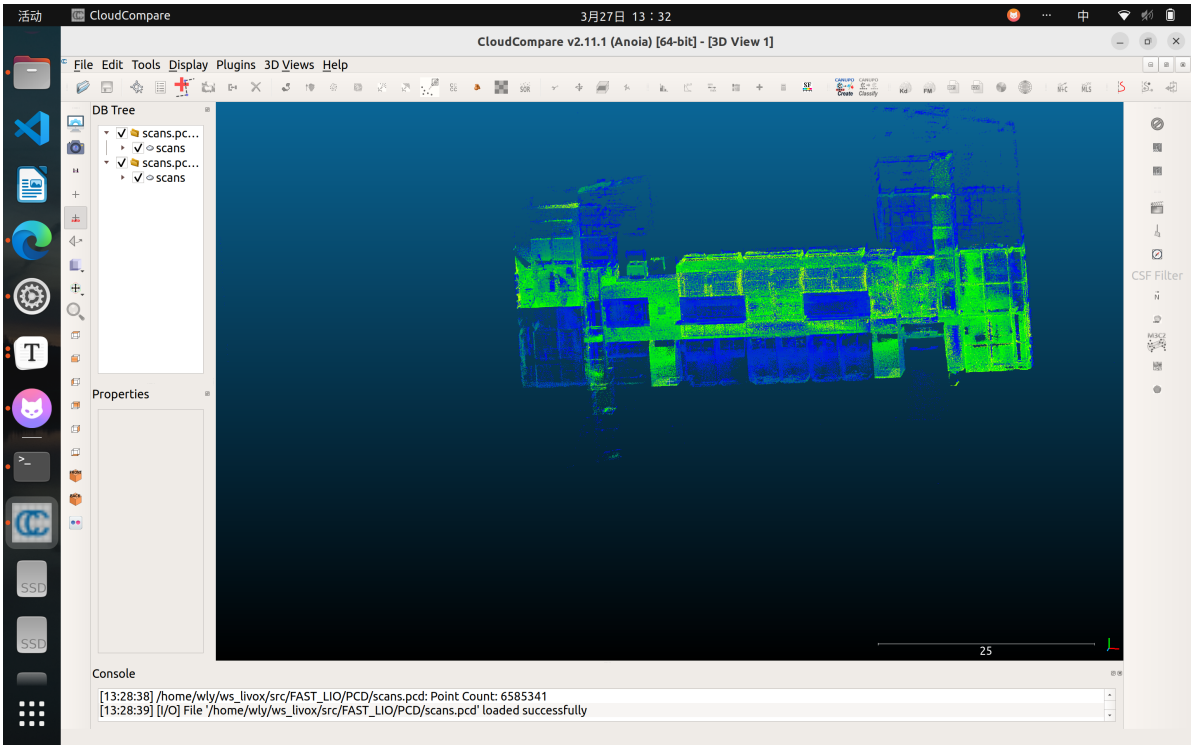
`rosbag play test.bag`



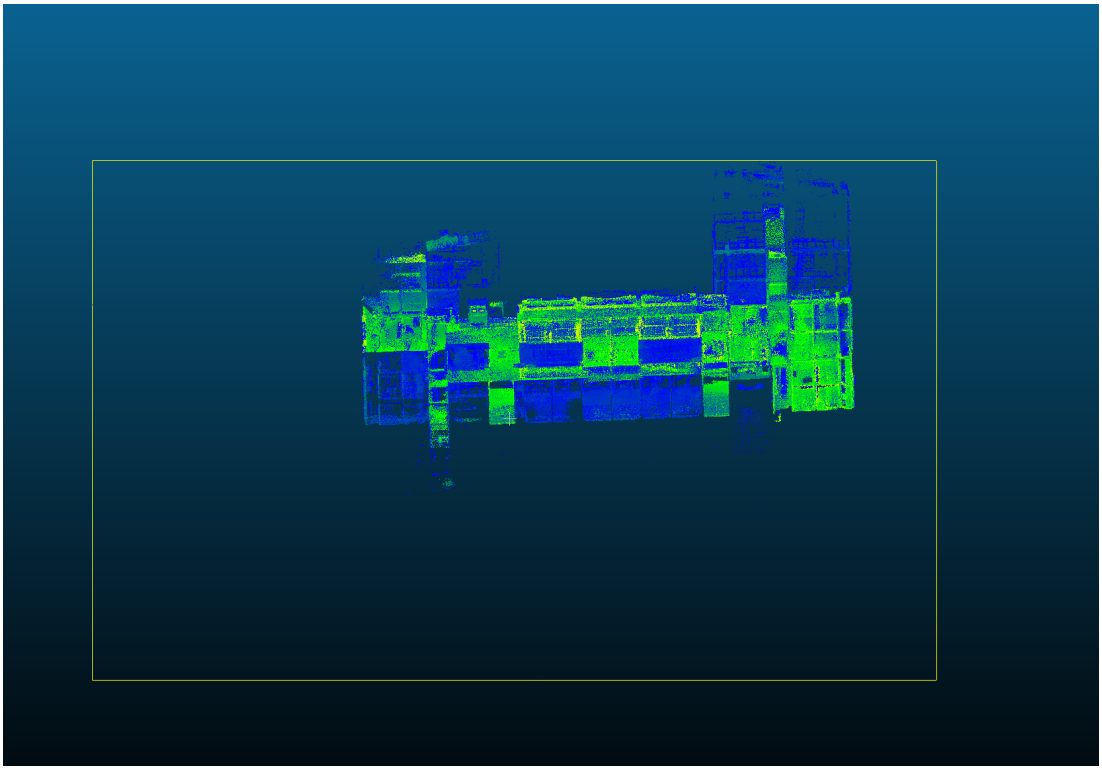
可以看到建图的过程；

当数据包播放完以后，`ctrl+c`，scans.pcd保存在目录PCD下

打开CloudCompare，查看图



看到地图有点歪斜，使用**lever工具校正**，



接下来是导出栅格地图，

安装**pcd2pgm**，打开test.cpp里面，找到对应的注释部分，修改对应路径和名称，编译，运行

`roscore`

`rosrun pcd2pgm pcd2topic`

`rosrun map_server map_saver`

现在pgm和yaml文件都生成了

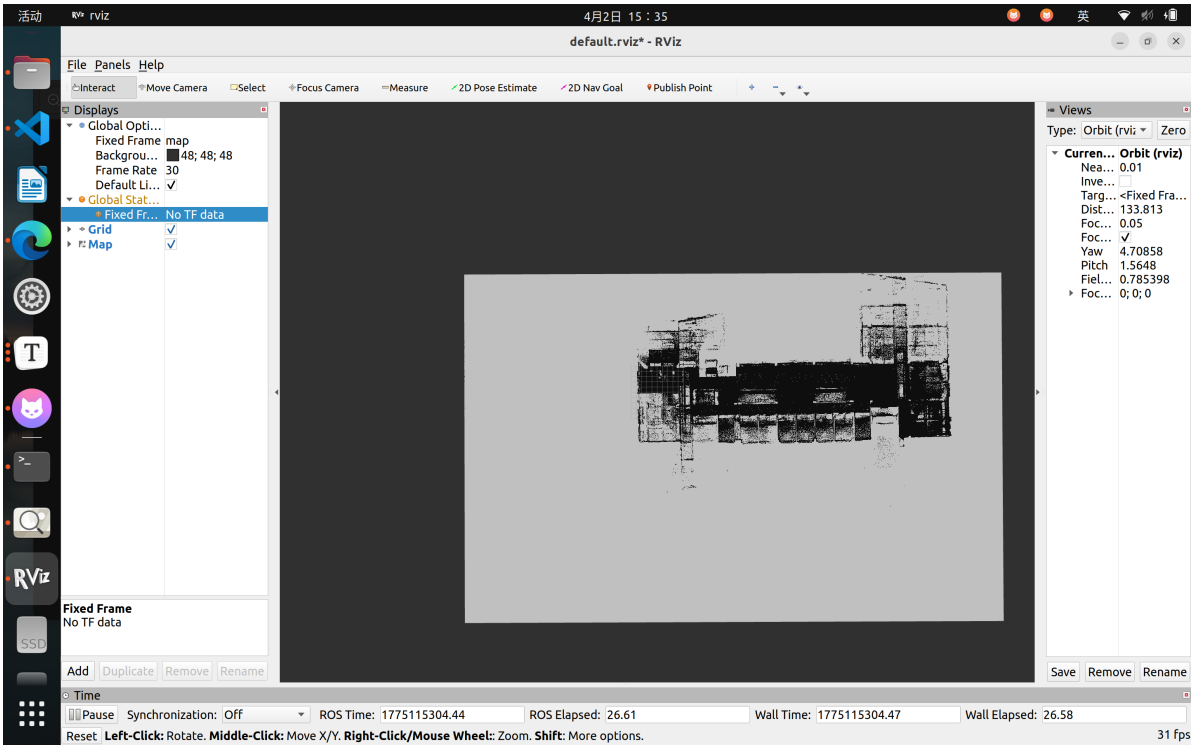
用rviz加载：

`rosrun map_server map_server scans.yaml`

`rviz`

显示地图





3.更加适配动态的建图算法

1. **LIO算法的缺点：**

   静态优先具体体现：

   --- 计算残差公式中默认全局地图中预先存在的点时恒定不变的，以此来修正当前位姿

--- 维护地图逻辑中，只要收敛就会被转换到全局坐标系，合并到tree，但并没有删除机制，即默认一旦出现就永远存在，因此肯出现重影；

--- 当前面物体和机器人相对静止移动，IMU识别在移动，激光雷达识别不动，二者会产生漂移

2. **解决方案：引入动态环境处理机制**

- **动态点剔除**：即把动态点去除，只用静态点建图
    - **几何一致性检测类**：如果上一帧有点，而下一帧没有了，那么为动态物体；

      实现：Ray Casting（射线投射）||Visibility Check（视点可见性）||当前帧 和全局地图重叠区域比较

      常见算法： **ERASOR, Removert**

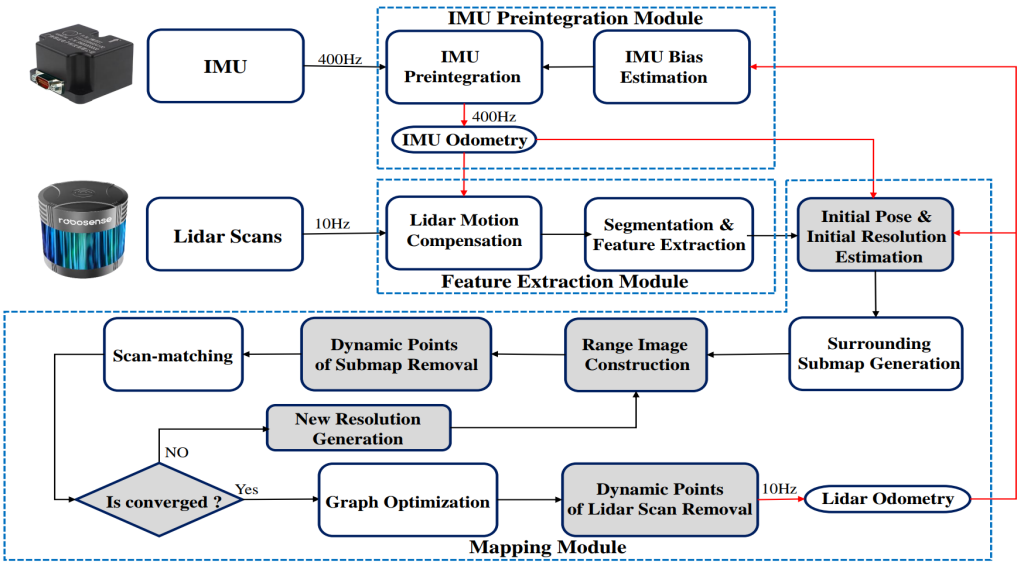
      优点&缺点：能检测物体消失，适用场景多；但是对位姿要求精度高，慢速运动不敏感
    - **语义分割类：**在点云输入 IEKF 之前，先经过一个语义分割模型，标记为动态的点在前端删掉

      常见算法：**SuMa++, SA-LOAM**

      优点&缺点：精度高，但是可能出现误检，且对未知类别无法处理
    - **端到端与时空耦合**：LIO预测下一帧的位置，如果预测正确，则认为是一个可追踪的动态物体，参与地图更新，反之则剔除；（时空一致性）
        - 常见算法：**RF-LIO / SLICT2**
        - 优点&缺点：不会误删；

RF-LIO

> Removal-First Lidar Inertial Odometry,剔除优先的激光惯导里程计



- **原理：**
    - **流程：**

      IMU预积分得到预测下一帧位姿，把**去畸变**（同fast-lio）后的点云投影到球面坐标系，生成2D图像，同时把上一时刻维护的局部子图按照IMU预测的位姿投影统一视角，生成参考图，**对比二者的像素值**，当前扫描距离显著小于地图距离说明出现了物体；显著大于说明原来物体离开了；以此标记动态点并剔除；

从上一步得到的静态点云进行**特征提取**，得到边缘和平面特征（是绝对静止的点具有空间一致性）；

将提取出的特征投射到局部子图，计算特征点到子图对应平面或直线的残差，通过不断调整位姿使得静态特征点与地图结构完美重合；

将配准后的激光里程计结果、高频 IMU 预积分量以及回环检测信息作为因子插入到全局**因子图**中（ IMU 因子约束短时间内的连续性，激光因子提供空间定位，回环因子负责消除长时间运行带来的累计漂移）；利用最小二乘法对这些约束进行全局求解，更新机器人的位姿、速度以及 IMU 零偏；当位姿优化收敛后，将当前帧处理好的静态特征点转换到全局坐标系，合并到全局地图中；

根据第一步可见性检查，如果判定某个地图点所在的区域物体移开，**回溯并删除**地图中那些点。最终产生一张**静态 3D 地图**，进行下一帧匹配；

回环检测；

- **关键：动静分离**

  利用几何关系（视点可见性）来定位动态点；

  在提取平面特征前完全剔除了所有动态的点云，使得特征点完全是静态的，保证了空间一致性；

  对全局地图会进行更新删除，保证了时间一致性；

Fast_LIO+YOLO

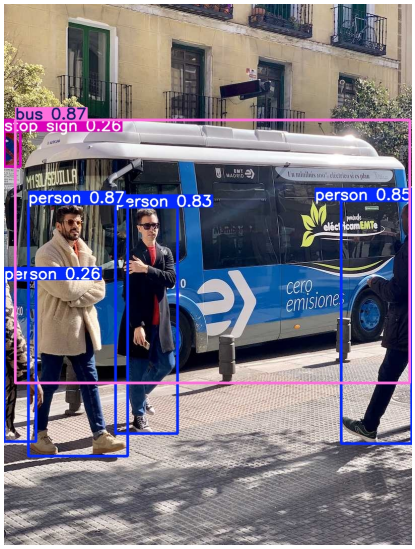
- **原理：基于投影的实时语义滤波**

  在点云输入 IEKF 之前，将其转换成2d的距离图像交给yolo,通过特征提取和生成掩码，对比原始点云，去除掉动态的物体，然后执行下面的步骤；

  - 转换:点云去畸变（利用 Fast-LIO2 内部的 IMU 预积分数据，对原始点云进行运动补偿）；设定分辨率，球面投影；
  - 识别物体：距离图归一化，调用YOLOv8-seg TensorRT，输出标签，提取掩码
  - 去除：遍历原始点云，同时找到2d投影图的坐标和标签，存入点云结构体此时点云具备了语义；遍历，如果标签是动态物体就剔除；
- 流程：

1. 简单配置并测试yolov8

（笔记本是集显，使用`openvino`模型）



2. 创建一个新包，包里面写yolo识别的脚本，这里用python示例，如下：（AI生成）

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rospy
import numpy as np
import cv2
from ultralytics import YOLO
from livox_ros_driver2.msg import CustomMsg, CustomPoint

class LivoxSemanticFilter:
    def __init__(self):
        rospy.init_node('livox_semantic_filter_node', anonymous=True)

        # 1. 加载 OpenVINO 模型 （这里使用了绝对路径）
        model_path = rospy.get_param('~model_path',
'/root/catkin_ws/src/my_semantic_filter/scripts/yolov8n-seg_openvino_model/')

        try:
            self.model = YOLO(model_path)
            rospy.loginfo(f"Successfully loaded model from {model_path}")
        except Exception as e:
            rospy.logerr(f"Failed to load model: {e}")
            return

        # 2. 定义 Mid-360 投影参数
        self.H = 64
        self.W = 1024
        self.fov_up = 7.0
        self.fov_down = -52.0
        self.fov_range = self.fov_up - self.fov_down

        # 3. 订阅与发布
        self.sub = rospy.Subscriber("/livox/lidar", CustomMsg, self.callback,
queue_size=1)
        self.pub = rospy.Publisher("/livox/lidar_filtered", CustomMsg,
queue_size=1)
        rospy.loginfo("Semantic Filter Node [CustomMsg Mode] Started")
```

```
    def project_to_range_image(self, points):
        """将点云投影为 Range Image"""
        x, y, z = points[:, 0], points[:, 1], points[:, 2]
        r = np.sqrt(x**2 + y**2 + z**2)

        # 避免除以 0
        r[r < 0.1] = 0.1

        yaw = np.arctan2(y, x)
        pitch = np.arcsin(z / r)

        # 映射到像素坐标 (u, v)
        u = (0.5 * (1 - yaw / np.pi) * self.W).astype(int)
        v = ((1 - (np.degrees(pitch) - self.fov_down) / self.fov_range) *
self.H).astype(int)

        # 边界检查
        u = np.clip(u, 0, self.W - 1)
        v = np.clip(v, 0, self.H - 1)

        # 生成简单的距离图
        range_image = np.zeros((self.H, self.W), dtype=np.float32)
        range_image[v, u] = r

        # 归一化并转为 3 通道灰度图供 YOLO 使用
        max_r = np.max(range_image) if np.max(range_image) > 0 else 1.0
        img_normalized = (range_image / max_r * 255).astype(np.uint8)
        img_3ch = np.stack([img_normalized]*3, axis=-1)

        return img_3ch, u, v

    def callback(self, msg):
        # 1. 快速将 CustomMsg 转为 NumPy 矩阵处理
        point_num = len(msg.points)
        if point_num == 0:
            return

        # 提取 x, y, z 进行几何计算
        points_xyz = np.zeros((point_num, 3), dtype=np.float32)
        for i in range(point_num):
            points_xyz[i, 0] = msg.points[i].x
            points_xyz[i, 1] = msg.points[i].y
            points_xyz[i, 2] = msg.points[i].z

        # 2. 投影
        img, u_coords, v_coords = self.project_to_range_image(points_xyz)

        # 3. YOLO 推理
        # imgsz 必须与导出模型时一致
        results = self.model.predict(source=img, imgsz=[self.H, self.W],
device='cpu', verbose=False)

        # 4. 创建掩码并过滤
        mask_final = np.zeros((self.H, self.W), dtype=bool)
        for result in results:
            if result.masks is not None:
                # 合并所有被识别为动态物体的 mask
                m = result.masks.data.cpu().numpy()
```

```
                combined_mask = np.any(m, axis=0)

                # 确保 mask 大小与投影图一致
                if combined_mask.shape != (self.H, self.W):
                    combined_mask = cv2.resize(combined_mask.astype(float),
(self.W, self.H)) > 0.5
                mask_final = np.logical_or(mask_final, combined_mask)

        # 5. 生成过滤后的消息
        # 这种方式通过复用原始点的对象来保留 offset_time 等关键信息
        filtered_msg = CustomMsg()
        filtered_msg.header = msg.header
        filtered_msg.header.frame_id = "livox_frame" # 确保与 Fast-LIO2 坐标系一致
        filtered_msg.timebase = msg.timebase
        filtered_msg.lidar_id = msg.lidar_id

        # 判定点是否保留：点投影位置的 mask 为 False 则保留
        new_points = []
        for i in range(point_num):
            if not mask_final[v_coords[i], u_coords[i]]:
                new_points.append(msg.points[i])

        filtered_msg.points = new_points
        filtered_msg.point_num = len(new_points)

        # 6. 发布
        self.pub.publish(filtered_msg)

if __name__ == '__main__':
    try:
        node = LivoxSemanticFilter()
        rospy.spin()
    except rospy.ROSInterruptException:
        pass
```

这段代码完成了以下任务：

订阅雷达原始点云----执行投影、YOLO 推理、生成 Mask-----发布一个新的话题（`/livox/lidar_filtered`这是去掉了动态点的干净点云）

3. 接下来下载 yolov8n-seg.pt 并将其转换为 OpenVINO 格式

`yolo export model=yolov8n-seg.pt format=openvino half=True imgsz=64,1024`

这行命令的意思：

`model=yolov8n-seg.pt`：这是原始模型

`format=openvino`：指定导出为 Intel CPU 加速格式

`imgsz=64,1024`：针对 Mid-360 的长条形投影图进行尺寸定制（高度 64，宽度 1024）

进入文件夹，能看到以下三个核心文件：

1. `yolov8n-seg.xml`：模型的结构图

2. `yolov8n-seg.bin`：模型的权重数据
3. `metadata.yaml`：存储类别信息

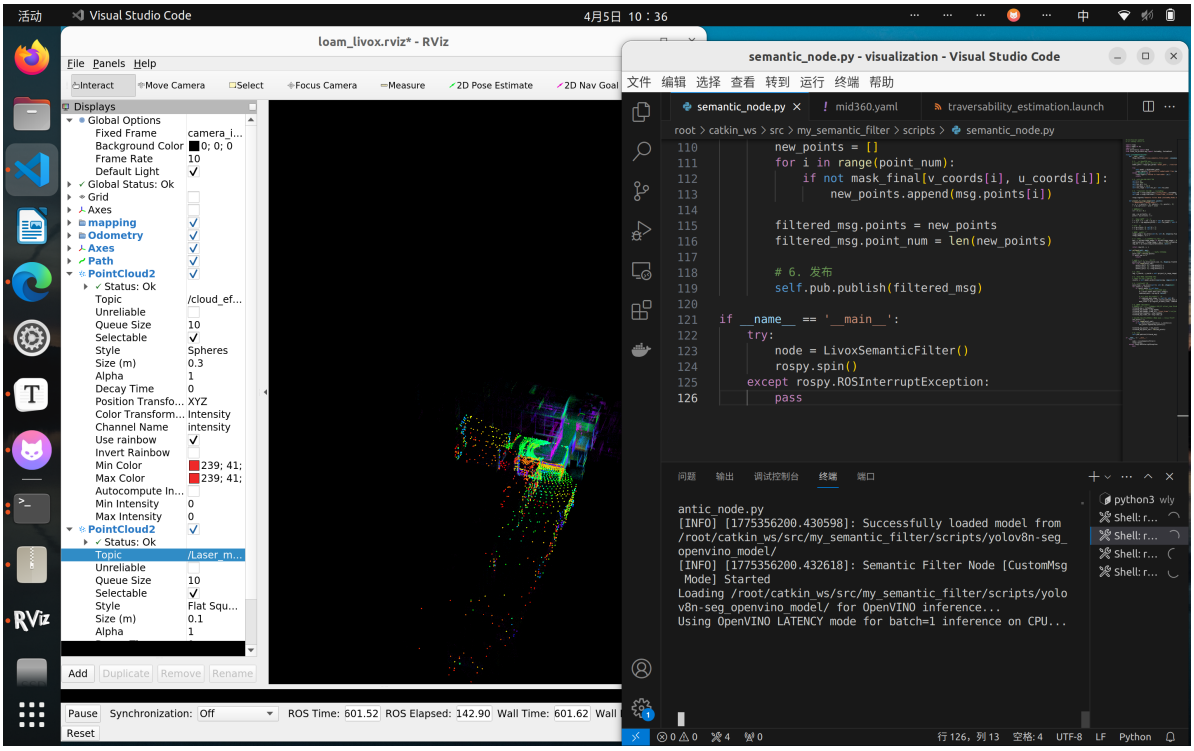
---

4. **修改fast_lio数据接受话题**

打开 `FAST_LIO/config/mid360.yaml`，把 `lidar_topic` 从 `/livox/lidar` 改为 `/livox/lidar_filtered`

所以最终数据流：

1. **数据包/雷达** 发布 `/livox/lidar`
2. **新节点** 订阅 `/livox/lidar` **运行 YOLO** 发布 `/livox/lidar_filtered`
3. **Fast-LIO** 订阅 `/livox/lidar_filtered` **构建干净的地图**



4.利用高维地图实现导航

1. **2D栅格地图**

将三维世界投影成一张黑白像素图，默认地面平坦，且所有障碍物都垂直于地面，机器人只需2D 像素中寻找最短路径，在像素间寻找代价最小的折线或曲线；

2. **2.5D 高程地图**

在每个 x, y坐标上存储一个 **z 轴高度数据**，引入了**坡度**和**粗糙度**和**台阶高度**计算，引入了**可通行性评估**（计算代价，寻找代价最低路线）；导航planner依据代价，**优先选择平坦低坡度区域**

（本质上是依据增加的z轴方向的数据，计算每个栅格的代价变成是否可以通行，输出一张2d代价地图，导航规划在2d进行）

- **流程架构：**

  激光雷达输出3D点云---

  `elevation_mapping`节点在水平面 (x, y)上建立固定分辨率的概率层，通过**卡尔曼滤波**融合不同时刻扫描到的高度值 z，维护一个**均值高度图**（刻画地形起伏）和**方差图**（不确定性）---

  `grid_map`计算法向量，坡度，台阶高度，输入到**Traversability**评估模型，生成权重**代价地图**，发布话题---订阅 `grid_map` 发布的 `traversability`话题---

  `move_base`在搜索路径时，会读取每一个格点的代价，规划路径

  在`robot_filter_parameter.yaml`中可以看到：可以根据实际机器人调整阈值

```
# Traversability map filter parameters
traversability_map_filters:
  - name: surfaceNormalsFilter #(表面法向量滤波器)
    type: gridMapFilters/NormalVectorsFilter
    params:
      input_layer: elevation
      output_layers_prefix: surface_normal_
      radius: 0.05
      normal_vector_positive_axis: z
  - name: slopeFilter
    type: traversabilityFilters/SlopeFilter #(坡度滤波器)
    params:
      map_type: traversability_slope
      critical_value: 1.0
  - name: stepFilter
    type: traversabilityFilters/StepFilter #(阶梯/台阶滤波器)
    params:
      map_type: traversability_step
      critical_value: 0.12
      first_window_radius: 0.04
      second_window_radius: 0.04
      critical_cell_number: 4
  - name: roughnessFilter
    type: traversabilityFilters/RoughnessFilter #(粗糙度滤波器)
    params:
      map_type: traversability_roughness
      critical_value: 0.05
      estimation_radius: 0.05
  - name: weightedSumFilter
    type: gridMapFilters/MathExpressionFilter
    params:
      output_layer: traversability
      expression: (1.0 / 3.0) * (traversability_slope + traversability_step +
traversability_roughness)
  - name: deletionFilter
    type: gridMapFilters/DeletionFilter
    params:
      layers: [surface_normal_x, surface_normal_y, surface_normal_z]
```

- **利用Turtlebot3简单模拟：**（简单使用house地图体验，虽然地面是平的🐱）

  1. **slam建图,保存地图**

  `export TURTLEBOT3_MODEL=waffle`

  `roslaunch turtlebot3_gazebo turtlebot3_house.launch`

启动 SLAM 建图：

```
roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=gmapping
```

键盘控制机器人建图，保存地图；

2. **启动高程建图**

先启动仿真gazebo，加载刚刚保存的2d地图，

```
roslaunch turtlebot3_navigation turtlebot3_navigation.launch map_file:=$HOME/map.yaml
```

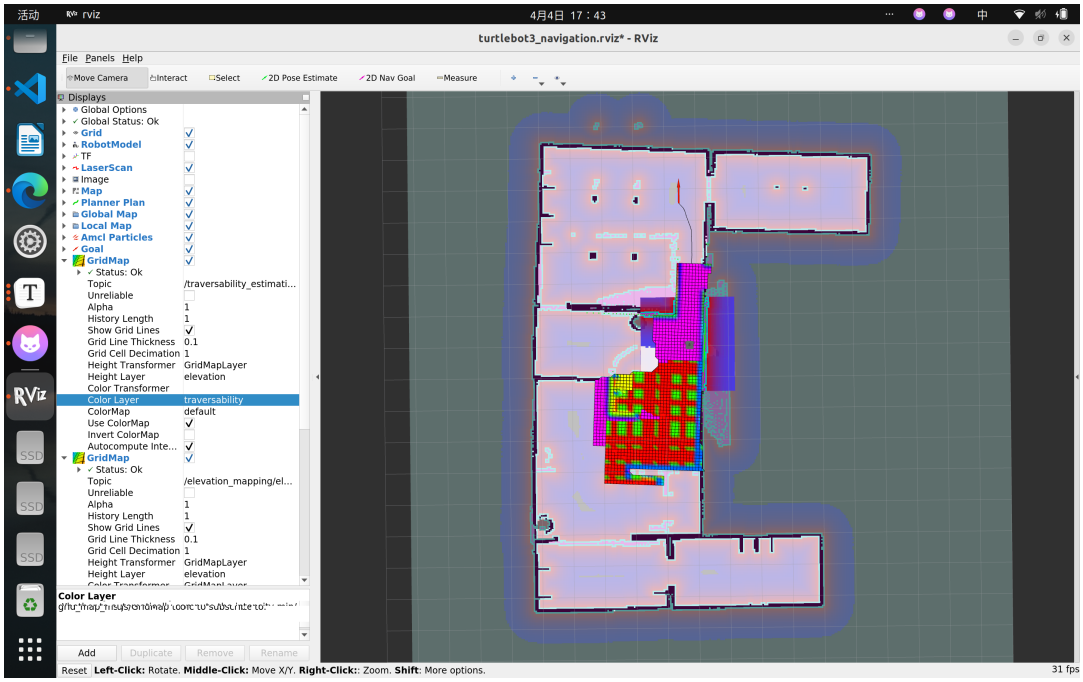
然后启动高程建图：

```
roslaunch elevation_mapping_demos turtlesim3_waffle_demo.launch
```

评估计算得出代价地图

```
roslaunch traversability_estimation_demos traversability_estimation_demo.launch
```

在rviz设定目标点导航



3. **3D概率占据地图**

接收 3D 点云数据，通过**贝叶斯概率**更新每个体素voxel（就是x,y,z;2d由像素组成是x,y）的**占用概率**（因为有噪声）;使用八叉树（Octomap）结构，从一个大立方体开始，内部有障碍物有空隙，就切成 8 个小立方体，直到设定的精度（实现了只有障碍物附近切分精细，节省内存）；增加了机器人的体积约束（设定高度区间，也就是机器人能通过的最大高度）

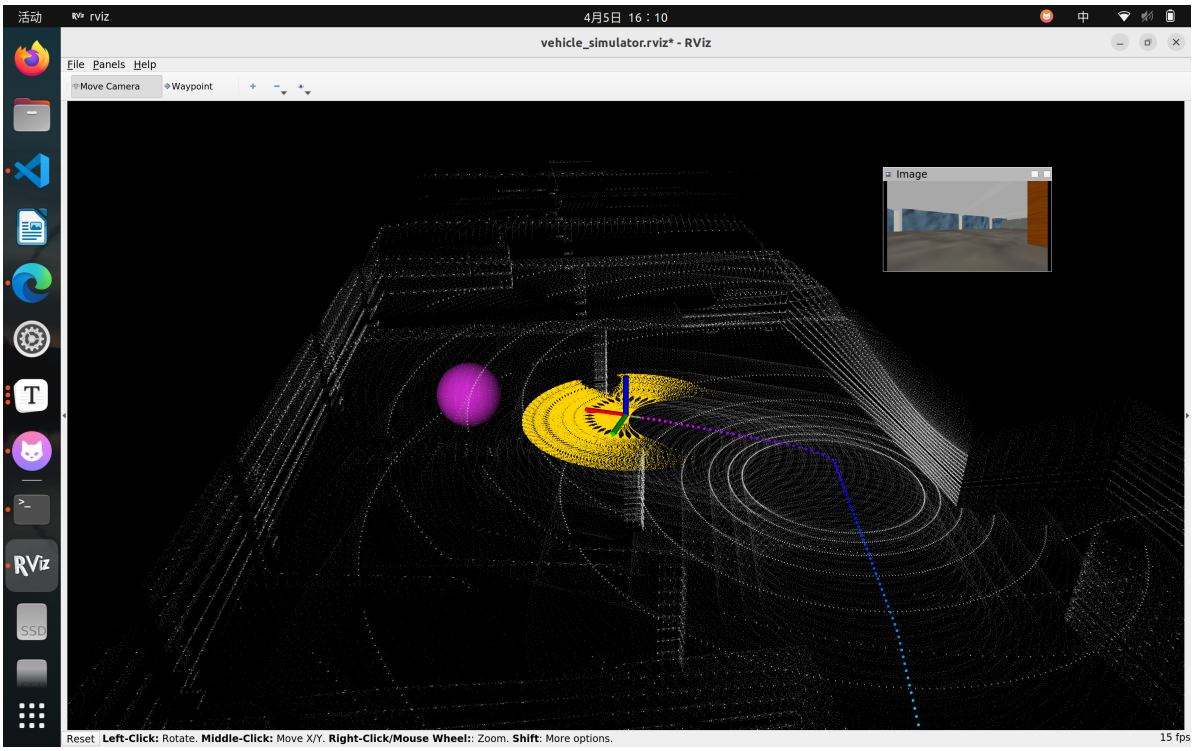
（本质是将空间障碍物占据的体素投影到xy平面，生成2d代价地图导航）

- 典例：
    1. **Octomap_server + move_base**：将 FAST-LIO 产生的 3D 点云 实时体素化，生成 **Octomap**，然后投影出一张 2D 占据栅格图 给 `move_base` 使用
    2. **Ego-Planner**：主要针对无人机，也可以移植到地面机器人；完全在 **3D 空间**内进行规划，支持动态避障，不依赖 `move_base`，而是使用 B 样条曲线（B-Spline）生成极度平滑的轨迹

3. **Terrain Analysis**：能分析 3D 点云中哪些地方是“可行驶的”，生成一个局部路径，引导机器人避开无法爬过的陡坡或坑洼

- **复现Terrain Analysis：**

克隆源码，按照教程，效果如下：



系统导航由三个层级组成：

- 局部规划与避障：Motion Primitives算法（预先生成一系列可能的行驶路径，实时检测哪些路径被障碍物遮挡并将其剔除，从剩余的无碰撞路径中优选出最接近目标的路径）
- 地形可通行性分析：局部图（实时分析地形平整度，计算每个点的 代价值）和扩展图（利用滑动窗口保留雷达历史点云，为高层规划器提供中长距离的地形信息）
- 航点跟踪：接收目标点、导航边界和期望速度，通过底层的控制器驱动机器人到达指定位置

4. **语义地图**

给几何体打上标，地形材质区分、动态预测

5. **动力学约束与模型预测**

实现了重心平衡、滑动预测；

导航时实时监测机器人的**重心**，增加防翻滚逻辑（在斜坡上行走时，限制转向角度和加速度）；规划一条带有时间轴的曲线；

总结

学到了什么🐾

首先，终于完成了考核！😆

通过将近一个月的时间，我深入理解了ROS Navigation Stack的整体架构，通过下载源码有了更深的理解，move_base作为导航核心整合全局规划器、局部规划器和代价地图，来完成从目标点到速度指令的完整流程；区分了`global_costmap`（map坐标系下使用静态地图进行长距离路径规划）和`local_costmap`（在odom坐标系下采用滑动窗口进行实时避障）；理解了DWA局部规划和D*全局规划各自的优势和如何配合使用；

在机器人建模方面，我学习了URDF和Xacro的语法，能够模仿并创建简单的机器人模型，在URDF中嵌入Gazebo仿真插件，让激光雷达能够发布/scan话题并且让差速驱动能够响应/cmd_vel指令；同时应用于后面的导航任务,我学会了配置costmap和planner；

在二维建图领域，我理解了Gmapping算法的原理与参数配置，理解了粒子滤波算法，同时通过写gmapping的launch文件知道了minimumScore，maxUrange和maxRange控制激光雷达有效探测范围，linearUpdate和angularUpdate控制地图更新的频率和用xmin、xmax、ymin、ymax限制建图边界等参数的功能；学会了map_server的使用方法，在建图完成后保存地图，在导航时加载地图；

在Gazebo仿真方面，我能够创建自定义的.world世界文件来搭建测试环境，使用spawn_model节点将机器人生成在指定位置；

在航点导航方面，从单点导航开始，封装成SimpleNavigator类，使用actionlib::SimpleActionClient与move_base进行通信，再到多点导航，vector存储航点坐标，用for循环遍历所有航点；并通过turtlebot3和自定义机器人,编写launch，有了更深入的了解；

在自主探索建图方面，我理解了基于边界检测的探索策略，即机器人不断检测前沿点，选择最优边界作为下一个目标点，当没有边界剩余时探索完成；之后下载了explore_lite包，将其与Gmapping和move_base联合启动测试建图，实现了发送目标点、move_base规划路径、机器人移动、激光雷达扫描、Gmapping更新地图、包检测新边界，循环直至探索完成的流程；

在三维建图方面，通过arxiv和github我深入学习了FAST-LIO一代和二代LIO的SLAM框架，并完成了复现工作；此外了解到LIO对于动态环境尚有欠缺，于是搜索更加适配动态的建图算法，了解到主流的三种算法，简单阅读了RF-LIO相关文献并总结，思考YOLO和fast-lio结合并进行简单实现；

在高程地图与3D地图方面，分别了解了各自的原理，对于高程地图没有找到合适的项目，简单使用turtlebot3测试；对于3d地图，对Terrain Analysis进行复现；

与此同时，在整个学习过程中，频繁地报错和测试使得我对rostopic list、rostopic echo、tf_echo和view_frames等工具的使用更加熟悉；我从一个初学者，逐步深入理解和学习，逐步掌握了搭建自定义机器人模型、在Gazebo中构建自定义仿真环境、运行Gmapping二维建图、配置move_base导航堆栈、实现航点循环导航和自主探索建图、并涉足FAST-LIO2三维建图、语义SLAM和高程地图的全过程。

从二维走向三维，从仿真走向真机，期待通过基地的考核，我会继续在机器人导航的道路上深耕不辍！🔥

遇到的问题以及解决方法❔

很大一部分是编译出现的问题，包括缺少包/版本不匹配等

常见的问题：C++标准版本冲突

然后是一些具体的问题，简单列举一些（过程中记录了一些但不全,暂时能想到的）：

1. urdf：joint 和link 顺序错误，显示异常
2. global_costmap与local_costmap共用同一参数文件，但需要区分static_map和rolling_window
3. 控制自定义的机器人时频繁卡住或者是撞墙不动：膨胀半径和摩擦系数设置不当（甚至开始忘记设置摩擦了）；调低`occdist_scale`（避障权重）；

4. gazobo搭建地图时没有注意坐标原点，导致最后的地图很偏，后面机器人出生点默认是原点，于是就到了图外面，需要重新指定出生点
5. 机器人生成位置错误：spawn_model的x/y/z参数与AMCL初始位姿不一致
6. 机器人转弯过于灵活：惯性参数使用默认值，没有结合实际修改
7. 建图过程中机器人突然漂移：轮式里程计累积误差
8. DWA的速度设置不合适：机器人要么太快要么太慢（最终喂给AI帮忙调）
9. 机器人倒着行走：尝试 换了插件里的 `leftJoint` 和 `rightJoint` 名字，不可行；改了DWA里面反向速度，也不行；最终发现电机旋转方向与 ROS 的 x轴正方向相反，修改 URDF 中轮子 `joint` 的 `<axis xyz="0 0 1"/>` 为 `0 0 -1`，同时将差速插件的 `odometrySource` 从 `world` 改为 `encoder`，确保 RViz 里的位姿变化完全由轮子转速决定，消除了 Gazebo 世界坐标系的干扰；
10. 用自己的机器人或地图导航需要配置launch以及config里的地图规划器，
11. 忘记重映射`scan`话题，gmapping订阅默认的`/scan`话题但没有数据，一直等待超时
12. FAST-LIO 源码默认适配livox_ros_driver一代，询问AI得到解决方法：

    修改 `package.xml` 和 `CMakeLists.txt`，将所有 `livox_ros_driver` 依赖改为 `livox_ros_driver2`；将源代码（`.cpp`, `.h`, `.hpp`）中的头文件引用和命名空间从 `livox_ros_driver::CustomMsg` 替换为 `livox_ros_driver2::CustomMsg`

    在 `CMakeLists.txt` 中添加 `add_dependencies`，强制要求先生成二代驱动的消息头文件，再编译算法节点
13. 在cloudcompare保存修正后的地图时，发现没有.pcd类型，于是保存为.asc，同过AI写的python 转换脚本得到.pcd文件
14. Docker容器中运行Gazebo无法显示GUI：需要配置`xhost +`和`--net=host`参数

AI使用情况

- 询问AI帮助解决以下类别问题：

帮忙用通俗的语言解释 插件机制/AMCL原理/代价地图/点云等概念

把自己对算法和概念的理解（比如对gmapping fast-lio的理解）发给AI看对不对，并让其帮忙补充

查询rosbag的使用方法

指出写的机器人urdf的错误，给出修正建议

结合出现的异常（机器人逆行等），问题可能出现在哪里？给出排查流程和解决方案

自定义机器人配置costmap和planner时总是撞墙，膨胀层如何调整

当前的最前沿的二维建图算法有什么，各自有什么特点进行不同维度对比

总结算法的发展流程（从最原始开始一步步增加了什么功能，优点是）

FAST-LIO 源码默认适配livox_ros_driver一代，mid-360雷达适配二代，询问AI得到解决方法：

对比二维栅格地图，高程地图和三维地图，列举一些当前热门的项目

生成表格等等

以及配置过程及复现过程中缺少了包/版本不对等问题，借助AI解决

附件：

压缩包文件结构，包括了任务二和三内自建图和自建机器人相关功能的实现

```
my_sim/
│
├── launch/
│   ├── display.launch                    # 加载自定义地图+生成自定义机器人
│   ├── my_sim.launch                     # 加载自定义地图+生成turtlebot3
│   ├── move_base.launch                  # 航点导航（自定义地图+机器人）
│   ├── my_gmapping.launch                # turtlebot3的gmapping配置
│   ├── dis_gmapping.launch               # 自定义机器人的gmapping配置
│   └── auto_explore.launch               # explore_lite建自定义图
│
├── urdf/
│   └── my_robot.urdf.xacro               # 机器人URDF模型（含激光雷达）
│
├── map/
│   ├── my_world/                         # Gazebo世界文件
│   │   └── my_world.world                # 仿真环境世界文件
│   ├── map.yaml                          # 导航用地图元数据
│   └── map.pgm                           # 导航用地图图片
│
├── config/
│   ├── costmap_common_params.yaml        # 通用代价地图参数
│   ├── global_costmap_params.yaml        # 全局代价地图参数
│   ├── local_costmap_params.yaml         # 局部代价地图参数
│   └── dwa_local_planner_params.yaml     # DWA局部规划器参数
├── rviz/
│   ├── nav.rviz                          # 导航视图配置
│   └── explore.rviz                      # 探索建图视图配置
│
├── src/
│   └── nav1.cpp                          # 航点导航程序
├── CMakeLists.txt
└── package.xml
```

